

**Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky**

**Vývoj rozšírenia prehliadača na detekciu
toxického obsahu pomocou NLP modelu**

Bakalárska práca

**Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky**

**Vývoj rozšírenia prehliadača na detekciu
toxického obsahu pomocou NLP modelu**

Bakalárska práca

Študijný program:	Informatika
Študijný odbor:	Informatika
Školiteľ:	Ing. Oliver Lohaj, PhD.
Školiace pracovisko:	Ústav umeleckej inteligencie (ÚUI)

Košice 2026

Vladyslav Stakhov

Abstrakt v SJ

Táto práca sa zameriava na detekciu toxického textu v prostredí webového prehliadača a na vývoj rozšírenia s názvom *Toxic Text Detector*. Cieľom práce je poskytnúť používateľom praktický nástroj na identifikáciu potenciálne škodlivého online obsahu pri zachovaní používateľskej kontroly a ochrany súkromia. Riešenie bolo navrhnuté a implementované ako rozšírenie prehliadača Chrome založené na architektúre Manifest V3. Využíva používateľské rozhranie vytvorené v Reacte, content script na prácu s textom webovej stránky, service worker na komunikáciu medzi komponentmi, perzistentné nastavenia a dva režimy inferencie. Lokálny režim vykonáva klasifikáciu priamo v prehliadači, zatiaľ čo vzdialený režim využíva externé API prostredníctvom backendovej služby. Implementované rozšírenie dokáže analyzovať vybraný text alebo textové prvky stránky, označiť potenciálne toxický obsah, vizuálne ho rozmazať a zobrazíť detegovanú kategóriu spolu so skóre istoty. Riešenie bolo vyhodnotené prostredníctvom testovania používateľského rozhrania a experimentov s modelmi zameraných na lokálnu a vzdialenú inferenciu. Výsledky ukazujú, že hybridný prístup integrovaný do prehliadača môže poskytnúť praktický kompromis medzi kvalitou detekcie, latenciou, použiteľnosťou a ochranou súkromia. Práca zároveň potvrdzuje, že detekcia toxicity zostáva náročná najmä pri menšinových kategóriách a kontextovo závislých formách škodlivého jazyka.

Kľúčové slová v SJ

detekcia toxicity, rozšírenie prehliadača, spracovanie prirodzeného jazyka, viac-
štítková klasifikácia, TensorFlow.js, Chrome Manifest V3, súkromie

Abstract

This thesis focuses on the detection of toxic text in a web browser environment and on the development of a browser extension named *Toxic Text Detector*. The aim of the work is to provide users with a practical tool for identifying potentially harmful online content while preserving user control and privacy. The solution was designed and implemented as a Chrome Manifest V3 extension. It uses a React-based user interface, a content script for working with web page text, a service worker for communication between components, persistent settings and two inference modes. The local mode performs classification directly in the browser, while the remote mode uses an external API through a backend service. The implemented extension can analyze selected text or text elements on a page, mark potentially toxic content, blur it visually and display the detected category together with a confidence score. The solution was evaluated through user interface testing and model experiments focused on local and remote inference. The results show that a hybrid browser-based approach can provide a practical compromise between detection quality, latency, usability and privacy protection. At the same time, the work confirms that toxicity detection remains challenging, especially for minority categories and more context-dependent forms of harmful language.

Key words

toxicity detection, browser extension, natural language processing, multi-label classification, TensorFlow.js, Chrome Manifest V3, privacy

Bibliografická citácia

STAKHOV, Vladyslav. *Vývoj rozšírenia prehliadača na detekciu toxického obsahu pomocou NLP modelu*. Košice: Technická univerzita v Košiciach, Fakulta elektrotechniky a informatiky, 2026. 61 s. Vedúci práce: Ing. Oliver Lohaj, PhD.

TECHNICKÁ UNIVERZITA V KOŠICIACH
FAKULTA ELEKTROTECHNIKY A INFORMATIKY
Katedra počítačov a informatiky

ZADANIE
BAKALÁRSKEJ PRÁCE

Študijný odbor: **Informatika**

Študijný program: **Informatika**

Názov práce:

Vývoj rozšírenia prehliadača na detekciu toxického obsahu pomocou NLP modelu

Development of a browser extension to detect toxic content using an NLP model

Študent: **Vladyslav Stakhov**

Školiteľ: **Ing. Oliver Lohaj, PhD.**

Školiace pracovisko: **Katedra kybernetiky a umelej inteligencie**

Konzultant práce:

Pracovisko konzultanta:

Pokyny na vypracovanie bakalárskej práce:

1. Analyzovať odbornú literatúru a existujúce riešenia v oblasti detekcie toxického obsahu v textoch a vývoja rozšírení webových prehliadačov.
2. Navrhnuť a implementovať metódu NLP na detekciu toxického obsahu v textoch vo forme multi-label klasifikácie.
3. Implementovať dva režimy inferencie, lokálny režim s využitím TensorFlow.js a vzdialený režim prostredníctvom externého API, a porovnať ich základné vlastnosti.
4. Vyvinúť rozšírenie webového prehliadača, ktoré integruje navrhnutú metódu a umožňuje používateľovi zobrazenie výsledkov detekcie a základné nastavenia správania systému.
5. Vypracovať dokumentáciu práce v súlade s pokynmi školiaceho ústavu a vedúceho práce.

Jazyk, v ktorom sa práca vypracuje: slovenský

Termín pre odovzdanie práce: 29.05.2026

Dátum zadania bakalárskej práce: 31.10.2025



prof. Ing. Liberios Vokorokos
prof. Ing. Liberios Vokorokos, PhD.
dekan fakulty

Čestné vyhlásenie

Vyhlasujem, že som záverečnú prácu vypracoval samostatne s použitím uvedenej odbornej literatúry, ako aj s využitím nástrojov generatívnej umelej inteligencie v súlade s Metodickým pokynom o záverečných a kvalifikačných prácach a Etickým kódexom študenta Technickej univerzity v Košiciach. Umelá inteligencia bola použitá ako podporný nástroj bez porušenia zásad akademickej etiky a autorskej integrity.

Podakovanie

Na tomto mieste by som sa rád poďakoval svojmu vedúcemu práce za jeho čas a odborné vedenie počas riešenia mojej záverečnej práce. Rovnako by som sa rád poďakoval svojim rodičom a priateľom za ich podporu a povzbudzovanie počas celého môjho štúdia.

Obsah

1	Úvod	1
2	Analytická časť	3
2.1	Teoretické základy	3
2.2	Prehľad súčasného výskumu	4
2.2.1	Sikiandani et al.: detekcia v prehliadači s BiLSTM a ASR	4
2.2.2	Kumar et al.: interpretovateľný klasický NLP pipeline (ToxiGuard)	5
2.2.3	Deep et al.: prehliadačový plug-in s multimodálnou detekciou (text + obraz)	6
2.2.4	Podporné artefakty: baseline, datasety a evaluačné protokoly	7
2.2.5	Syntéza: dôsledky pre návrh riešenia	8
2.3	Metódy a charakteristiky modelu	10
2.4	Etické a spravodlivé hľadiská	12
2.5	Architektúra navrhovaného riešenia	12
2.6	Metriky hodnotenia a testovanie	14
2.7	Diskusia o otvorených problémoch	14
2.8	Kalibrácia prahu a hodnotenie orientované na používateľa	14
2.9	Úvahy o viacjazyčnosti a prechode medzi jazykmi	15
2.10	Vývoj rozšírení v rámci Chrome Manifest V3	17
2.11	Experimentálny protokol a podávanie správ	17
3	Ciele záverečnej práce	19
4	Praktická časť	22
4.1	Úvod do praktickej časti	22
4.2	Požiadavky na riešenie	23
4.2.1	Funkčné požiadavky	23
4.2.2	Nefunkčné požiadavky	23
4.3	Návrh riešenia	24

4.3.1	Architektúra rozšírenia	25
4.3.2	Návrh režimov inferencie	26
4.3.3	Návrh toku dát	27
4.3.4	Návrh práce s kategóriami toxicity a prahmi	28
4.3.5	Bezpečnostné a súkromnostné opatrenia	28
4.4	Prototypovanie používateľského rozhrania	29
4.4.1	UX výskum	29
4.4.2	Návrh sekvencií obrazoviek	30
4.4.3	Prototypy používateľského rozhrania	31
4.4.4	Zmeny na základe prototypovania	32
4.5	Implementácia rozšírenia	33
4.5.1	Build proces a konfigurácia	33
4.5.2	Implementácia popup rozhrania	34
4.5.3	Implementácia stránky nastavení	34
4.5.4	Správa nastavení a perzistencia	35
4.5.5	Background logika a komunikácia	35
4.6	Implementácia lokálnej inferencie toxicity	36
4.6.1	Načítanie modelu a normalizácia výstupu	36
4.6.2	Klasifikácia textu a práca s cache	37
4.6.3	Vizuálne označenie toxického obsahu	38
4.7	Vzdialený režim inferencie cez API	38
4.8	Doplňujúce mechanizmy implementácie	38
4.8.1	Whitelist domén	39
4.8.2	Získavanie štatistík z aktívnej stránky	39
4.9	Praktické obmedzenia implementácie	39
4.10	Príprava experimentálneho overenia	40
4.11	Zhrnutie praktickej časti	40
5	Vyhodnotenie riešenia	42
5.1	Metodika vyhodnotenia	42
5.2	Vyhodnotenie používateľského rozhrania	43
5.2.1	Prvý prototyp a jeho verifikácia	43
5.2.2	Druhý prototyp a jeho verifikácia	44
5.3	Vyhodnotenie vzdialeného režimu inferencie	44
5.3.1	Nerovnováha tried a príprava dát	45
5.3.2	Iterácie modelu a porovnanie verzií	45
5.3.3	Prahy rozhodovania: tuned vs. product	46
5.4	Vyhodnotenie predtrénovaného TF.js toxicity modelu	47

5.4.1	Nastavenie experimentu	47
5.4.2	Výsledky a interpretácia	48
5.5	Vyhodnotenie optimalizovaného lokálneho režimu inferencie . . .	49
5.5.1	Nastavenie experimentu	49
5.5.2	Ladenie rozhodovacích prahov	50
5.5.3	Klasifikačné výsledky na validačnej množine	51
5.5.4	Porovnanie výkonu backendov CPU a WASM	52
5.6	Zhrnutie vyhodnotenia	52
6	Záver	54
	Zoznam použitej literatúry	56
	Zoznam príloh	62

Zoznam obrázkov

4.1	Konceptuálny model rozšírenia.	25
4.2	Návrh postupností obrazoviek a navigácie rozšírenia.	30
4.3	Prvý prototyp vo Figma.	31
4.4	Ukážka implementovaného rozhrania v druhom prototype.	32

Zoznam tabuliek

2.1	Porovnanie prác a artefaktov	9
5.1	Rozdelenie pozitívnych príkladov	45
5.2	Porovnanie verzií XLM-R	46
5.3	Prahy tuned vs product	46
5.4	Vyhodnotenie predtrénovaného TF.js toxicity modelu	48
5.5	Per-label výsledky predtrénovaného TF.js modelu	48
5.6	Nastavenie lokálneho Char-CNN experimentu	50
5.7	Prahy pre lokálny Char-CNN model	50
5.8	Per-label výsledky lokálneho Char-CNN modelu	51
5.9	Celkový toxický verdikt lokálneho modelu	51
5.10	Porovnanie CPU a WASM backendu	52

1 Úvod

Internet sa stal jednou z najväčších súčastí moderného ľudského života. Sociálne siete, textové fóra a sekcie komentárov dali každému používateľovi možnosť verejne vyjadriť svoj názor z ktorejkoľvek časti sveta. Táto dostupnosť však viedla k šíreniu škodlivého a toxického obsahu. Používatelia sú denne vystavení urážkam, nenávisťným prejavom, hrozbám a iným neprijateľným formám toxického obsahu, ktoré negatívne ovplyvňujú duševné zdravie.

Preto sa automatická detekcia takéhoto obsahu stala aktívnou oblasťou výskumu v oblasti umelej inteligencie a ešte viac v oblasti spracovania prirodzeného jazyka (NLP). Moderné prístupy využívajú hlboké učenie na identifikáciu toxicity s vysokou presnosťou a prispôbivosťou. Väčšina týchto systémov však beží na serverovej infraštruktúre, čo znamená, že niektoré osobné údaje používateľov musia byť prenesené na externé platformy, čo znižuje súkromie a bezpečnosť používateľov. Riešenie integrované do prehliadača s možnosťou lokálneho vyhodnocovania textu môže minimalizovať tieto riziká a zároveň zabezpečiť plynulý používateľský zážitok.

Cieľom tejto práce je vyvinúť a vytvoriť rozšírenie prehliadača, ktoré dokáže analyzovať vybraný text na webovej stránke a určiť jeho úroveň toxicity. Výsledok aplikácie sa zobrazí v rozhraní prehliadača s uvedením, či je obsah toxický alebo netoxický, alebo s číselným skóre toxicity. Systém by mal fungovať v dvoch režimoch:

1. **Lokálny režim**, s použitím vopred natrénovaného modelu bežiaceho v prehliadači.
2. **Vzdialený režim**, ktorý zavolá externé API na klasifikáciu.

Nedávny výskum ukazuje, že je to v praxi možné. Modely hlbokého učenia už dosahujú dobré výsledky pri detekcii bežných prejavov toxicity. Implementácia v reálnom svete však stále čelí niekoľkým prekážkam: nerovnováha tried (napr. hrozby sú menej bežné a ľahšie sa prehliadajú), jazyková a dialektová skreslenosť.

Obzvlášť náročné zostávajú jemnejšie formy zneužívania – ako je sarkazmus, kódovaný jazyk alebo kontextové urážky.

Formulácia úlohy

Hlavnou úlohou tejto práce je vytvoriť aplikáciu, ktorá používateľom poskytne jasný a rýchly spôsob, ako pochopiť, či je text na stránke škodlivý alebo nie. Rozšírenie prehliadača, ktoré analyzuje vybraný text a poskytuje jasný záver, pomôže používateľom robiť informované rozhodnutia v reálnom čase. Okrem pohodlia má aj ďalšiu výhodu – znižuje dopad toxických výmen. Toto rozšírenie tiež pomôže udržiavať konštruktívnosť online priestoru a zachovať bezpečnosť používateľov. Projekt sa nachádza na priesečníku webového inžinierstva a umelej inteligencie.

Táto práca rieši tieto problémy prostredníctvom dizajnu zameraného na používateľa a súkromie. Stručne povedané, rozšírenie ponúkne dva režimy výstupu – lokálny a vzdialený – bude prezentovať hodnotenia značiek a bude obsahovať možnosť hodnotiť výkon aplikácie, aby používatelia mohli nahlásiť chyby. Spoločne sa zameriava na vyváženie presnosti s jednoduchosťou a vytvára nástroj, ktorý je nielen efektívny, ale aj skutočne užitočný pri každodennom prehliadaní webu pre bežného používateľa.

2 Analytická časť

Táto kapitola predstavuje analytický základ navrhovaného riešenia na detekciu toxického textu v prostredí webového prehliadača. Najprv sú opísané teoretické východiská problému, následne sú analyzované existujúce prístupy, datasety, modely a evaluačné metriky. Kapitola sa ďalej venuje etickým a spravodlivým aspektom, architektúre navrhovaného riešenia, otázkam viacjazyčnosti, špecifikám Chrome Manifest V3 a experimentálnemu protokolu použitému pri hodnotení systému.

2.1 Teoretické základy

Termín *toxicita* online sa vzťahuje na výrazy, ktoré sú hrubé, neúctivé alebo môžu spôsobiť, že sa ostatní budú cítiť zle. Vo výskume sa problém bežne modeluje ako *viacnásobná* (*multi-label*) klasifikácia textu, kde jeden text môže patriť do viacerých kategórií, ako *toxicita*, *vysoká toxicita*, *obscénnosť*, *vyhrážka*, *urážka* alebo *nenávisť voči identite*. Referenčné dátové súbory—najmä séria Jigsaw—štandardizovali túto taxonómiu a slúžia ako referenčné body pre hodnotenie modelov [1][2][3][4][5][6][7][8].

Tradičné prístupy k trénovaniu systému detekcie toxicity vyžadujú milióny anotovaných komentárov. Tradičné systémy skutočne používali funkcie (bag-of-words) so štatistickými klasifikátormi, ako sú Support Vector Machines alebo Naïve Bayes. Hoci boli tieto tradičné metódy účinné, mali určité nedostatky: nedokázali hlboko porozumieť kontextu. Architektúry hlbokého učenia, ako napríklad LSTM, BiLSTM, CNN a neskôr transformátory, dosiahli v poslednej dobe skvelé výsledky vďaka svojej jasnej schopnosti zachytiť kontext a sémantický význam aj za hranicami izolovaných slov [1][2][9][10][11][12].

2.2 Prehľad súčasného výskumu

Existujúce práce o detekcii toxicity a škodlivého obsahu pre koncových používateľov možno rozdeliť do troch vrstiev:

1. Prehliadačovo integrované systémy, ktoré sa snažia detegovať škodlivý obsah priamo v mieste konzumácie (rozšírenia, plug-iny),
2. ľahké NLP pipeline, ktoré uprednostňujú transparentnosť a nízku výpočtovú náročnosť,
3. zdieľané artefakty (datasets, evaluačné metriky a produkčné baseline), ktoré štandardizujú štítky a umožňujú porovnateľné hodnotenie.

Toto rozdelenie vychádza z porovnania prehliadačových riešení, klasických NLP pipeline a spoločných výskumných artefaktov používaných pri hodnotení toxicity [6][7][9][10][13][14].

Pre túto prácu je podstatné nielen to, ako presný je model „na papieri“, ale aj celkový príbeh nasadenia: súkromie (lokálna vs. vzdialená inferencia), latencia, konzistentné správanie UI, vysvetliteľnosť a otázky spravodlivosti. Prehľad literatúry naznačuje, že kvalitný UX je v prehliadači reálne dosiahnuteľný, avšak spôsob, akým sa inferencia vykonáva, má zásadný vplyv na súkromie a dôveru používateľov [15][16][17][18][19].

Pre túto prácu je podstatné nielen to, ako presný je model „na papieri“, ale aj celkový príbeh nasadenia: súkromie (lokálna vs. vzdialená inferencia), latencia, konzistentné správanie UI, vysvetliteľnosť a otázky spravodlivosti. Prehľad literatúry naznačuje, že kvalitný UX je v prehliadači reálne dosiahnuteľný, avšak spôsob, akým sa inferencia vykonáva, má zásadný vplyv na súkromie a dôveru používateľov.

2.2.1 Sikiandani et al.: detekcia v prehliadači s BiLSTM a ASR

Sikiandani, Suarjaya a Putra navrhujú systém orientovaný na prehliadač, kde rozšírenie pre Chrome tvorí používateľskú vrstvu a inferencia je realizovaná na backendovej službe. Zámerom je podporiť používateľa priamo počas čítania obsahu a poskytnúť rýchlu informáciu o potenciálne škodlivom texte [10].

Dáta a nastavenie problému. Autori modelujú toxicitu ako viacnásobnú (multi-label) klasifikáciu so štandardnou šesť-kategóriovou taxonómiou (toxic, severe

toxic, obscene, insult, threat, identity hate). Trénovacie dáta obsahujú približne 15 000 označených komentárov a vykazujú typickú nerovnováhu tried, kde zriedkavé štítky (napr. threat alebo severe toxic) môžu byť ľahko „prekryté“ dominantnými kategóriami [10].

Model a technológia. Textový klasifikátor je založený na BiLSTM a autori experimentujú s rôznymi embeddingmi. Dôležitým rozšírením je spracovanie reči v krátkych videách: audio sa najprv prepíše pomocou Whisper (ASR) a až následne sa prepis hodnotí z hľadiska toxicity. BiLSTM tu predstavuje pragmatickú voľbu: je ľahší než veľké transformery, no dokáže zachytiť viac kontextu než klasické bag-of-words prístupy [10][20].

Silné stránky. Práca ukazuje realistický používateľský scenár: detekcia prebieha priamo v prehliadači, výsledky sú zrozumiteľne prezentované a systém je rozširiteľný aj na audio obsah. Z praktického hľadiska je relevantné aj to, že ľahšie neurónové architektúry môžu dosiahnuť vysokú mieru spomienky (recall), čo je dôležité pri minimalizácii falošne negatívnych výsledkov [10].

Obmedzenia a riziká. Najväčším problémom je súkromie: inferencia je vzdialená, takže používateľský text (a potenciálne aj prepisy) opúšťa zariadenie. Navyše vzniká reťaz chýb, keďže klasifikátor vidí iba prepis a chyby ASR znižujú kvalitu následnej detekcie. Pri menšinových triedach môže systém pôsobiť silno na agregovaných metrikách, no stále podávať slabší výkon práve v kritických kategóriách [10][20].

Implikácie pre túto prácu. Táto štúdia podporuje koncept rozšírenia s UX vzormi typu „zamaskovať a odhaliť“ obsah, no zároveň jasne ukazuje potrebu lokálneho režimu, ak má byť riešenie privacy-by-default. Z pohľadu architektúry tým posilňuje motiváciu pre hybridný prístup: lokálna inferencia ako predvolená možnosť a vzdialená inferencia iba ako voliteľná alternatíva (napr. pri viacjazyčnosti alebo pre presnejší model).

2.2.2 Kumar et al.: interpretovateľný klasický NLP pipeline (ToxiGuard)

Kumar a kol. navrhujú ToxiGuard, ktorý kladie dôraz na interpretovateľnosť a efektivitu. Tento smer je zaujímavý najmä tam, kde je potrebná nízka latencia, jednoduché vysvetlenie výsledkov a použiteľnosť ako baseline pri porovnávaniach [9].

Pipeline a technológie. Riešenie využíva klasické predspracovanie NLP a bag-of-words príznaky, pričom klasifikátorom je Naïve Bayes (napr. cez CountVectorizer). Typický postup zahŕňa čistenie textu, tokenizáciu a normalizačné kroky, následne extrakciu príznakov a klasifikáciu [9].

Silné stránky. Takýto prístup je výpočtovo nenáročný, rýchly a transparentnejší než hlboké neurónové modely, čo môže zvyšovať dôveru používateľov v situáciách, kde očakávajú jednoduché zdôvodnenie. Zároveň je vhodný ako interpretovateľný baseline pri evaluácii a pri debugovaní správania systému [9].

Obmedzenia a riziká. Klasické bag-of-words modely typicky zlyhávajú pri kontexte, sarkazme a implicitnej nenávisti, keďže význam často závisí od slovosledu, negácií a dlhších závislostí. Z pohľadu nasadenia tiež nejde primárne o „extension-native“ riešenie, takže je potrebné odlišovať medzi výskumnou pipeline a reálnym prehliadačovým produktom [9].

Implikácie pre túto prácu. ToxiGuard je pre nás dôležitý najmä ako interpretovateľný referenčný bod a ako inšpirácia pre UI prvky typu „prečo“ (transparentnosť výsledku). Pre realistickú detekciu nuansovaných foriem toxicity a multi-label skórovanie však bude potrebný silnejší model než čistý Naïve Bayes baseline.

2.2.3 Deep et al.: prehliadačový plug-in s multimodálnou detekciou (text + obraz)

Deep a kol. navrhujú zásuvný modul prehliadača, ktorý analyzuje obscénny obsah v texte aj v obrázkoch, teda multimodálne. Takýto prístup je relevantný, pretože reálny web často kombinuje text s vizuálnym obsahom (memy, screenshoty), ktorý môže byť tiež škodlivý [13].

Silné stránky. Hlavným prínosom je rozšírenie perspektívy: prehliadačové nástroje nemusia byť obmedzené iba na text, ale môžu riešiť viac typov obsahu. To podporuje myšlienku, že používateľská ochrana sa dá integrovať priamo do toku prehliadania webu [13].

Obmedzenia a riziká. Analýza obrázkov zásadne zvyšuje výpočtovú náročnosť a zvyčajne prináša dodatočné oprávnenia a súkromnostné riziká, pretože obrázky

môžu obsahovať osobné údaje aj vtedy, keď text nie. V praxi je preto multimodálna náročnejšia na bezpečný a zodpovedný dizajn [13].

Implikácie pre túto prácu. Táto práca zostáva zámerne zameraná na toxicitu v texte, aby bolo hodnotenie kontrolované a aby sa uprednostnil lokálny režim zachovávajúci súkromie. Multimodálny smer však predstavuje prirodzený priestor pre budúce rozšírenia a potvrdzuje prenositeľnosť UX vzorov (blokovanie/odhaľovanie, vysvetlenie, spätná väzba).

2.2.4 Podporné artefakty: baseline, datasety a evaluačné protokoly

Okrem jednotlivých systémov je výskum toxicity silne formovaný zdieľanými artefaktmi. Tieto artefakty definujú taxonómie označení, poskytujú tréningové a testovacie údaje a zavádzajú metriky, ktoré odhaľujú spôsoby zlyhania skryté za jedným hlavným skóre.

Produkčný baseline: Perspective API. Perspective API poskytuje viac atribútov toxicity a je dlhodobo udržiavané rozhranie pre hodnotenie textu. Je vhodné na porovnanie, avšak ide o vzdialené API, takže text sa odosiela do externej služby a privacy-by-default je možné dosiahnuť iba pri explicitnom opt-in a jasnej komunikácii [14].

Benchmarky a taxonómia: Jigsaw datasety. Séria datasetov Jigsaw štandardizovala multi-label taxonómiu a poskytuje referenčné súbory na tréning a testovanie. Multilingual varianta zároveň ukazuje, že modely trénované iba na angličtine sa mimo angličtiny výrazne zhoršujú, ak nie sú explicitne prispôbosené [6][8].

Spravodlivosť: Unintended Bias a skupinové metriky. Dataset *Unintended Bias* zavádza identitne anotované testovanie a metriky orientované na podskupiny, ktoré odhaľujú neúmyselné skreslenia (napr. nadmerné označovanie identitných zmienok alebo dialektu). V kontexte nasadenia ide o kritický aspekt, pretože priamo ovplyvňuje dôveru používateľov a môže marginalizovať skupiny [7][19][21][22].

Vysvetliteľnosť: HateXplain. HateXplain obsahuje triedy, anotácie cieľovej skupiny a ľudské racionály (zvýraznené úseky textu), čím podporuje vysvetliteľnosť

a cieľné hodnotenie. Aj keď systém nemusí byť priamo trénovaný na racionáloch, dataset motivuje UI prvky typu „prečo“ a diskusiu o spoľahlivosti vysvetlení [18].

Robustnosť: ToxiGen. ToxiGen cieľi na implicitnú a adversariálnu nenávisť a poskytuje veľký dataset vhodný na stres-testovanie robustnosti modelov. Je užitočný najmä pri analýze zlyhaní, ktoré sa v reálnom online prostredí vyskytujú vo forme kódovaného jazyka alebo sarkazmu [23][24].

Lokálna inferencia v prehliadači: TF.js *toxicity* a MV3. Model TF.js *toxicity* demonštruje, že multi-label inferencia je realizovateľná lokálne v prehliadači. Pri praktickom nasadení však treba rátať s obmedzeniami MV3 (načítanie, pamäť, responzivita UI a bezpečnostné pravidlá) [15][16].

2.2.5 Syntéza: dôsledky pre návrh riešenia

Zhrnutie. Literatúra ukazuje, že prehliadačovo integrované UX vzory sú realizovateľné a často efektívne, no vzdialená inferencia znižuje súkromie a zvyšuje nároky na transparentnosť. Klasické pipeline sú užitočné ako interpretovateľné baseline, ale samy osebe nestačia na nuansovanú alebo implicitnú toxicitu. Zdieľané datasety a fairness-orientované metriky sú nevyhnutné, pretože agregované skóre často neodhaľuje zlyhania menšinových tried ani neúmyselné skreslenia [7][19][21][22].

Dopad na architektúru práce. Tieto zistenia priamo motivujú architektúru zvolenú v tejto práci: rozšírenie Chrome Manifest V3 s lokálnym režimom inferencie zachovávajúcim súkromie, voliteľným vzdialeným režimom cez bezpečný proxy, per-label skórovaním s nastaviteľnou prísnosťou a hodnotiacim protokolom, ktorý kombinuje prediktívne metriky (makro/mikro F1, Hammingova strata, výkon po štítkoch) s inžinierskymi metrikami (latencia, čas načítania modelu) a s fairness diagnostikou tam, kde je to relevantné [7][15][16][19].

Dielo / Artefakt	Model / prístup	Nasadenie	Dáta / poznámka
Sikiandani et al.	BiLSTM (multi-label) + Whisper ASR	Rozšírenie + vzdialený backend	~15k; 6 štítkov; text+reč; citlivé na menšinové triedy
Kumar et al. (ToxiGuard)	CountVectorizer/ BoW + Naïve Bayes	Výskumný nástroj (typ. lokálne)	Interpretovateľný baseline; slabší kontext/implicitná toxicita
Deep et al.	Multimodálne (text+obraz), hybrid	Plug-in prehliadača	Obscénny obsah; vyššia náročnosť a riziká pri obrázkoch
Perspective API	Proprietárne (transformery)	Vzdialené API	Multi-atribúty + jazyky; produkčný baseline, nie privacy-by-default
Jigsaw <i>Unintended Bias</i>	Evaluačné metriky + dataset	Offline hodnotenie	Subgroup AUC, BPSN/BNSP; identity značky (bias audit)
HateXplain	Dataset (racionály, ciele)	Offline dataset	~20k (Gab/Twitter); podporuje vysvetliteľnosť
ToxiGen	Dataset (implicitná/adversariálna)	Offline dataset	274k viet; stres-test implicitnej nenávisťi; 13 skupín
TF.js <i>toxicity</i>	Lahký multi-label model (TF.js)	Lokálne v prehliadači	Jigsaw-štýl štítkov; per-label skóre; vhodné pre privacy režim

Tabuľka 2.1: Stručné porovnanie prác a artefaktov relevantných pre detekciu toxicity (model, nasadenie, dáta).

2.3 Metódy a charakteristiky modelu

Predspracovanie. Typické kroky zahŕňajú prevod na malé písmená (ak veľkosť písmen nenesie dôležitú informáciu), Unicode normalizáciu (napr. NFKC), nahradenie URL adries, používateľských mien a hashtagov špeciálnymi symbolmi, zjednotenie opakovanej interpunkcie a mierne „čistenie“ textu. Pri detekcii toxicity je užitočné: (a) ponechať emotikony a emoji (nesú informáciu o náladе), (b) normalizovať leetspeak a medzery vo vulgárnych výrazoch a (c) zachovať negácie. Pri použití tokenizérov pre transformátory sa agresívne odstraňovanie stop-slov a stemming zvyčajne vynecháva; pri klasických modeloch môžu byť slovné a znakové n -gramy (napr. 3–5 znakov) veľmi robustné voči obchádzaniu filtrov [9][18][25].

Reprezentácia príznakov. Existujú tri praktické úrovne:

1. *Bag-of-words* / *TF-IDF* nad slovnými a znakovými n -gramami – rýchle, interpretovateľné a dobrý baseline [1][9].
2. *Statické vektory* (word2vec/GloVe/FastText) – vhodné pre CNN/BiLSTM, pričom FastText lepšie zvláda preklepy [25][26][27].
3. *Kontextové vloženia* (BERT/RobERTa, DistilBERT, mBERT/XLM-R pre viac-jazyčnosť) – dosahujú najlepšiu presnosť vďaka modelovaniu kontextu, náznakom sarkazmu a dlhých závislostí. Kompaktné alebo destilované varianty sú vhodnejšie pre použitie v prehliadači [6][8][11][12][23][28][29].

Klasifikátory. Základné modely, ako logistická regresia alebo lineárne SVM, fungujú dobre s TF-IDF reprezentáciou. V neurónových modeloch sa často používajú CNN (rýchle, zachytávajú lokálne vzory), BiLSTM/GRU (sekvenčné modelovanie, často s pozornosťou) a *hybridné* architektúry (napr. CNN+BiLSTM). Transformátory doladené na dátach toxicity zvyčajne dosahujú najvyššiu presnosť. BiLSTM systém opísaný Sikiandanim a kol. ukazuje, že kompaktné rekurentné modely môžu dosiahnuť vysokú mieru spomienky pri nízkych výpočtových nákladoch, čo je atraktívne v prostrediach s obmedzenými zdrojmi [1][9][10][11][12][28].

Cieľ multi-label učenia. Štandardnou voľbou straty je *binárna cross-entropia na každú značku*. Nerovnováha tried (napríklad to, že *hrozba* je veľmi zriedkavá)

sa rieši pomocou *váh tried*, *focal loss* alebo vyváženého vzorkovania. Veľmi účinnou a jednoduchou pákou je použitie *prahov osobitne pre každú značku* (namiesto jedného globálneho prahu), ktoré sa doladia na validačných krivkách presnosť–spomienka [5][7][30][31].

Kalibrácia a používateľské nastavenia. Okrem optimalizácie F1 je užitočné kalibrovať pravdepodobnosti (napr. pomocou *temperature scaling* alebo Plattovej kalibrácie), aby skóre lepšie zodpovedali úrovni istoty, ktorú UI zobrazuje. V samotnej aplikácii môže jeden posuvník prístnosti posúvať všetky prahy naraz, pričom relatívne rozdiely medzi značkami zostanú zachované, a panel s výsledkami by mal vždy zobrazovať *aj* diskretný verdikt, *aj* skóre pre jednotlivé značky [32].

Hodnotenie. Je potrebné reportovať *presnosť*, *spomienku* a *F1* pre každú značku, *makro-F1* (všetky značky majú rovnakú váhu), *mikro-F1* (zohľadňuje objem), *Hammingovu stratu* (chybovosť podľa štítkov) a *PR-AUC* pre porovnanie nezávislé od konkrétneho prahu. Súčasťou by mali byť aj *konfúzne matice* pre menšinové značky (napr. Threat, Severe toxic). Pre rozšírenie do prehliadača sú rovnako dôležité aj *inžinierske metriky*: latencia (medián/p95 od výberu textu alebo načítania stránky po verdikt), čas prvého načítania modelu a pamäťová stopa pri lokálnom behu [5][6][7][15][16][19][21][31][33][34].

Robustnosť a viacjazyčnosť. Je vhodné doplniť záťažové testy pre bežné obchádzky (medzery, leetspeak, vkladanie symbolov) a spúšťať *bias probes* na frázach s identitnými a dialektálnymi výrazmi. Pri zmiešaných alebo neanglických stránkach pomôže ľahká detekcia jazyka a smerovanie na viacjazyčný model; prahy podľa jazyka (lokality) pomáhajú zabrániť nadmernému označovaniu v menšinových jazykoch [8][19][22][24][29].

Vysvetliteľnosť (voliteľná, ale užitočná). Pri neurónových modeloch môžu jednoduché vizualizácie na úrovni tokenov (napr. pozornosť alebo gradientové mapy) pomôcť recenzentom pochopiť, prečo bol konkrétny úsek textu označený. Takéto výstupy by však mali byť prezentované ako *heuristiky*, nie ako presné vysvetlenia rozhodnutia modelu [18].

Inžinierske poznámky (lokálny vs. vzdialený režim). Pri *lokálnom* behu v TF.js je vhodné voliť kompaktné alebo destilované modely, obmedziť maximálnu dĺžku sekvencie, dávkovať kratšie úryvky a kešovať výsledky pre opakovane sa vyskytujúci text. Pri *vzdialenom* behu je dôležité používať proxy (kde sa uchováajú

API kľúče), nastavovať limity požiadaviek a v rozhraní jasne komunikovať, keď je text odosielaný mimo zariadenia používateľa [15][16][28].

Zhrnutie. V praxi sa ako veľmi pragmatická skladba ukazuje: ľahké predspracovanie → transformátor (alebo BiLSTM/CNN pre kompaktnejšie nasadenie) → prahy pre jednotlivé značky → verdikt + skóre pre značky, hodnotené pomocou makro-F1, Hammingovej straty a latencie, doplnené o testy zaujatosti a viacjazyčné fallback riešenia tam, kde sú potrebné.

2.4 Etické a spravodlivé hľadiská

Napriek vysokej agregovanej presnosti môžu automatizované detektory vykazovať problémy so spravodlivosťou. Štúdie a benchmarky poukazujú na to, že modely môžu nadmerne označovať dialektálne texty alebo zmienky o identite ako toxické, čo znižuje dôveru a môže marginalizovať určité skupiny; benchmark *Unintended Bias* sa týmto účinkom venuje priamo. Spravodlivosť a transparentnosť sú preto kľúčové. Navrhované riešenie zobrazí skóre dôvery pre jednotlivé štítky a umožní používateľom poskytovať spätnú väzbu pri nesprávnej klasifikácii; túto spätnú väzbu možno následne použiť na prekalibráciu prahov alebo na preučenie klasifikátora [7][19][21][22][32].

Z pohľadu súkromia lokálna inferencia zabezpečuje, že vybraný text nikdy neopustí zariadenie používateľa. Pre prípady, ktoré vyžadujú vyššiu presnosť alebo viacjazyčné pokrytie, možno použiť vzdialený režim *len so súhlasom* a s jasným informovaním, v súlade so zásadami minimalizácie a transparentnosti údajov podľa GDPR [15][16][17].

2.5 Architektúra navrhovaného riešenia

Architektúra rozšírenia bude vychádzať zo štandardov Chrome Manifest V3. Tento formát je v súčasnosti odporúčaný pre nové rozšírenia, pretože prináša prísnejší bezpečnostný model (žiadny vzdialene hostovaný kód, prísnejšia Content Security Policy) a nahrádza trvalé background pages udalosťami riadeným service workerom. To je výhodné pre náš projekt, pretože inferencia modelu môže prebiehať na pozadí bez blokovania používateľského rozhrania, pričom štruktúra MV3 prirodzene rozdeľuje zodpovednosti medzi jednotlivé časti rozšírenia [16][35][36][37][38].

V rámci rozšírenia budú:

- **Content skripty** zodpovedné za prístup k DOM – zachytávajú používateľom vybraný text a pri zapnutej funkcii aj automaticky skenujú text na stránke po načítaní [37].
- **Servisný pracovník** (pozadie), ktorý zabezpečuje komunikáciu s modelom, batchovanie požiadaviek a prepínanie medzi lokálnym a vzdialeným režimom inferencie [36][38].
- **Popup a bočný panel** (side panel), ktoré zobrazujú výsledky klasifikácie – hlavný verdikt (toxický / netoxický) spolu s čiastkovými skóre pre jednotlivé triedy toxicity.

Kľúčovým rozhodnutím je podpora dvoch režimov inferencie NLP modelu:

1. **Lokálny režim (TensorFlow.js)** Predtrénovaný model sa zabalí priamo do rozšírenia a beží v prehliadači pomocou TensorFlow.js. Tento režim poskytuje okamžitú spätnú väzbu, nevyžaduje internetové pripojenie a najmä chráni súkromie – analyzovaný text nikdy neopustí zariadenie používateľa. Zodpovedá aj zneniu zadania, ktoré explicitne spomína možnosť lokálneho modelu v prehliadači [15].
2. **Vzdialený režim (API cez bezpečný proxy)** Pre viacjazyčné scenáre a zložitejšie modely, ktoré je náročné bežať priamo v prehliadači, bude k dispozícii vzdialený režim. Rozšírenie v tomto prípade nekomunikuje priamo s poskytovateľom modelu (napr. HuggingFace Inference API alebo Perspective API), ale s vlastným jednoduchým backendom – bezpečným proxy. Proxy na serveri vkladá API kľúče, zabezpečuje obmedzenie počtu požiadaviek a umožňuje prípadné anonymizovanie alebo logovanie chýb. Tým sa predchádza tomu, aby citlivé kľúče boli uložené v kóde rozšírenia a aby bolo možné flexibilne meniť poskytovateľa modelu bez aktualizácie klienta [14][35].

Voľba tejto hybridnej architektúry (MV3 + lokálny TF.js model + vzdialené API cez proxy) vychádza z kombinácie požiadaviek zadania a cieľov práce: rozšírenie musí byť implementované v JavaScripte, malo by preferovať súkromie a rýchlosť (lokálny režim), ale zároveň umožniť experimentovať s presnejšími a viacjazyčnými modelmi (vzdialený režim). Takýto prístup je v súlade so zásadou minimalizácie údajov podľa GDPR, pričom vzdialená inferencia je explicitne voliteľná a používateľ o nej dostáva jasnú informáciu. Porovnanie oboch režimov – z hľadiska presnosti, latencie a dopadu na súkromie – bude tvoriť súčasť analytického hodnotenia v ďalšej časti práce [17].

2.6 Metriky hodnotenia a testovanie

Testovanie bude založené na referenčných dátových súboroch: *Jigsaw Toxic Comment Classification Challenge*, *Unintended Bias* a *Multilingual Toxic Comment*. Pre všetky tieto súbory budú uvedené údaje o presnosti, spomienke (recall), F1-skóre a makropriemere na značku; súčasťou bude aj *Hammingova strata* pre porovnateľnosť multi-label modelov. Testovanie bude zahŕňať aj porovnanie latencie a využitia zdrojov v lokálnom a vzdialenom režime. Matice zmätkov (confusion matrices) budú zobrazovať chyby v menšinových značkách a voliteľné kontroly zaujatosti pomôžu preskúmať rozdiely medzi jazykovými variantmi [5][6][7][8][10][15][16][19][22][24][33][34].

2.7 Diskusia o otvorených problémoch

Hoci najnovšia literatúra ukazuje solídny pokrok v tejto oblasti, stále zostáva niekoľko otvorených otázok bez odpovede:

1. Detekcia implicitnej a sarkastickej toxicity, ktorá vyžaduje pochopenie kontextu a nielen doslovného textu [6][23][24].
2. Zláhčovanie menšinových tried, ako sú *hrozby* alebo *závažná toxicita* [5][10].
3. Predsudky v rôznych jazykoch a nárečiach, ktoré obmedzujú spravodlivé používanie modelu [7][8][19][22].
4. Nasadenie spôsobom, ktorý chráni súkromie, pri zvažovaní kompromisu medzi presnosťou a etickým spracovaním údajov (lokálne spracovanie verus vzdialená inferencia v súlade s MV3 a GDPR) [15][16][17].

Navrhovaná práca sa snaží riešiť aspoň niektoré z týchto výziev implementáciou hybridného systému zameraného na ochranu súkromia, ktorý vyvažuje výkon a dôveru používateľov.

2.8 Kalibrácia prahu a hodnotenie orientované na používateľa

Keďže ide o problém s viacerými označeniami, jediný globálny prah môže viesť k nevýhodným kompromisom (napr. vynikajúca detekcia *toxicity*, ale mnoho prehliadnutých *hrozieb*). Vhodnejší prístup je použiť *prahy pre každé označenie*

nastavené na validačnom rozdelení tak, aby vyvažovali presnosť a spomienku pre každú kategóriu z taxonómie Jigsaw. V praxi to zahŕňa vykreslenie kriviek presnosť–spomienka pre každú značku a výber prevádzkových bodov, ktoré odrážajú ciele používateľského rozhrania (napr. prísnejšie posudzovanie *nenávisťi voči identite* a citlivejšie nastavenie pre *hrozby*). Aby rozhranie zostalo zrozumiteľné, rozšírenie môže zobrazovať jeden *hlavný verdikt* (“Pravdepodobne toxický”) a rozbaľovací zoznam skóre pre jednotlivé značky (napr. Toxický 0,91, Urážka 0,78, Hrozba 0,12), čím sprístupní pravdepodobnosti atribútov pri zachovaní produktových prahov [6][5][7][8][9][10][31][32].

Kalibrácia sa ďalej spája s *Hammingovou stratou* a *makro-F1*: Hammingova strata rovnako penalizuje každú chybu podľa označenia, a preto je užitočná na porovnanie lokálneho a vzdialeného režimu, aj keď sú ich profily chýb odlišné. Makro-F1 zachováva viditeľnosť menšinových kategórií priemerovaním F1 naprieč značkami. Vykazovanie *makro-F1* aj *Hammingovej straty* zosúladzuje hodnotenie s predchádzajúcou prácou a zabraňuje kozmetickým vylepšeniam, ktoré maskujú regresie menšinových značiek [5][6][10][19][33][34].

2.9 Úvahy o viacjazyčnosti a prechode medzi jazykmi

Webový text často predstavuje zmes jazykov, dialektov a transliterácií – napríklad latinizácia v nelatinských jazykoch alebo zmiešavanie angličtiny so slovenčinou v jednej vete. Porovnávacie štúdie, ako napríklad Jigsaw Multilingual challenge, preukázali, že modely, ktoré sú trénované výlučne na angličtine, dosahujú výrazne horšie výsledky v iných jazykoch, pokiaľ nie sú výslovne prispôsobené alebo vyladené pre tieto jazyky. V praxi to vedie k dvom kľúčovým problémom: nedostatočné odhalenie toxicity v neanglickom texte a nadmerné označovanie neškodných identitných termínov v jazykoch alebo dialektoch, ktoré model dostatočne nepozná [7][8][18][22][23][29].

Praktickým plánom rozšírenia je preto explicitne zaobchádzať s viacjazyčnosťou, namiesto toho, aby sa predstieralo, že model je „jazykovo agnostický“:

- **Lokálny režim: zameraný na angličtinu.** V tomto režime rozšírenie udržiava malý model zameraný hlavne na angličtinu. Je to preto, lebo spustenie v prehliadači obmedzuje veľkosť modelu: menšie modely sa načítajú rýchlejšie, vyžadujú menej pamäte a celé spracovanie zostáva na zariadení používateľa. Na oplátku akceptujeme, že iné jazyky ako angličtina budú v tomto režime podporované len obmedzene. Toto obmedzenie môže byť v rozhraní používateľa označené malou poznámkou, napríklad „opti-

malizované pre angličtinu“, aby používateľ vedel, kedy môžu byť výsledky menej spoľahlivé [8][15][28].

- **Vzdialený režim: viacjazyčné eskalovanie.** Keď ľahký detektor jazyka – alebo atribút `lang` stránky – indikuje, že vybraný text je prevažne v inom jazyku ako angličtina, rozšírenie prejde na viacjazyčný model prostredníctvom vzdialeného API. V tomto prípade sa vedľa výsledku zobrazí diskretný indikátor, napríklad „(viacjazyčný režim)“, aby používateľ vedel, že jeho text bol odoslaný na server a že sa používa iný model. Toto smerovanie môže spracovávať aj zmiešané prípady: ak je odsek čiastočne v angličtine a čiastočne v inom jazyku, rozšírenie ho môže rozdeliť na segmenty a odoslať iba segmenty, ktoré nie sú v angličtine, do vzdialeného modelu, pričom zvyšok zostane lokálny [8][14][29].
- **Prahové hodnoty špecifické pre jazyk/písmo.** Dokonca aj v prípade viacjazyčných modelov nie sú rozdelenia skóre v jednotlivých jazykoch identické. Aby sa zabránilo systematickému nadmernému alebo nedostatočnému označovaniu, systém udržiava samostatné prahové hodnoty pre každý jazyk alebo písmo, napríklad angličtina verzus slovanské jazyky, latinka verzus cyrilika. Tieto prahové hodnoty je možné nastaviť offline na základe validačných údajov alebo malých kurátorských testovacích súborov a uložiť v jednoduchvej vyhľadávacej tabuľke vnútri rozšírenia. Ide o veľmi lacný mechanizmus, ktorý sa zvyčajne oplatí v podobe výrazne zlepšenej vnímanej kvality [7][8][19].

Viacjazyčné problémy vznikajú aj nepriamo v procese ASR→klasifikácia, kde kvalita transkripčného modelu – napríklad Whisper – stanovuje hornú hranicu detekcie toxicity v nadväzujúcich procesoch. Dokonca aj silný viacjazyčný ASR môže mať problémy s rôznymi prízvukmi, prepínaním kódov a menovanými entitami; v takýchto prípadoch sa chyby v transkripcii prenášajú priamo do klasifikátora. Z tohto dôvodu by akákoľvek funkcia založená na reči v rozšírení mala výslovne hlásiť kvalitu transkripcie (alebo aspoň varovanie pre segmenty s nízkou spoľahlivosťou) a rozhodnutie o toxicite a vyhnúť sa prezentovaniu štítkov založených na ASR ako rovnako spoľahlivých ako štítky natívne pre text [10][20].

Celkovo nie je cieľom úplne vyriešiť detekciu viacjazyčnej toxicity, ale skôr zviditeľniť a kontrolovať výber jazyka: lokálne spracovanie anglického textu z dôvodu rýchlosti a ochrany súkromia, eskalované spracovanie neanglického textu na silnejšom viacjazyčnom back-ende, ak to používateľ povolí, a ladenie prahov pre každý jazyk s cieľom znížiť systematickú zaujatosť namiesto použitia jedného

globálneho prahu.

2.10 Vývoj rozšírení v rámci Chrome Manifest V3

MV3 prináša službu riadenú udalosťami namiesto pretrvávajúcej stránky na pozadí, prísnejšiu politiku CSP a zakazuje vzdialene hostovaný vykonateľný kód—čo ovplyvňuje načítanie modelu aj prístup k API [16][35][36]. Kľúčové vzory:

- **Skripty obsahu len pre DOM.** Skripty obsahu spracúvajú prístup k DOM a zachytávanie výberu; náročnejšia práca (lokálna inferencia, dávkovanie, volania API) sa deleguje na service worker cez odovzdávanie správ [16][37][38].
- **Balenie a lazy-loading modelu.** Zabaľte TensorFlow.js a artefakty modelu ako prostriedky rozšírenia a načítavajte ich pri prvom použití analýzy [15][16][35].
- **Bezpečnosť kľúčov pre vzdialený režim.** Nikdy nedistribuuajte poskytovateľské kľúče v klientskom balíku; použite minimálny proxy backend, ktorý vkladá poverenia na strane servera a aplikuje obmedzenie rýchlosti; rozšírenie drží len krátkodobý anonymný token [16][35].
- **Tipy na výkon.** Ukladajte do vyrovnávacej pamäte (*chrome.storage.session*) nedávne výsledky, sledujte čas a pamäťovú stopu načítania TF.js grafu a optimalizujte prvé použitie [15][16].

Tieto voľby zabezpečujú, že rozšírenie zostane odozvvové, kompatibilné s MV3 a bude podporovať lokálny aj vzdialený režim inferencie [15][16].

2.11 Experimentálny protokol a podávanie správ

Reprodukovateľný protokol spája technické rozhodnutia systému s merateľnými výsledkami:

- **Rozdelenie údajov.** Použiť stratifikované rozdelenie tréningových/validačných/testovacích dát v *Jigsaw Toxic Comment*, a hodnotiť generalizáciu medzi dátovými sadami na *Unintended Bias* a *Multilingual* (zero-shot, ak je tréning iba v angličtine) [6][7][8].
- **Metriky.** Reportovať presnosť (precision), spomienku (recall) a F1 na značku, makro-F1, Hammingovu stratu a zahrnúť matice zámien (confusion matrices) pre menšinové značky *Threat* a *Severe toxic* [5][6][10][31][33][34].

- **Metriky latencie a UX.** Merať čas od výberu používateľa po vykreslenie verdiktu (medián/p95) pre *lokálny* vs. *vzdialený* režim; zaznamenať čas načítania modelu pri prvom použití a sledovať pamäťovú stopu grafu TF.js [15][16].
- **Ablácie.** Porovnať s/bez normalizácie; *per-label* vs. *globálny* prah; lokálne vs. vzdialené spracovanie na rovnakých textoch; a viacjazyčné eskalácie na neanglických stránkach [6][7][8][15].
- **Sondy zaujatosti.** Použiť malé, ručne kurátorské sady s dialektálnymi výrazmi a zmienkami o identite na kvantifikáciu rozdielov a usmernenie úprav prahov [7][19][22][24].
- **Reč.** V sprievodnom experimente podať do textového klasifikátora prepisy z Whisper a reportovať *WER* prepisov spolu s metrikami toxicity, aby bol zviditeľnený reťazec chýb [10][20].

3 Ciele záverečnej práce

Celkovým cieľom tejto práce je vývoj, implementácia a experimentálne testovanie doplnku prehliadača, ktorý pomôže používateľom identifikovať toxický obsah na webových stránkach spôsobom zachovávajúcim súkromie, berúc do úvahy realizmus online interakcií. Mal by používateľom umožniť získať informatívne odpovede týkajúce sa toxicity vybraného textového úseku s rozumným kompromisom medzi čisto klientskym spracovaním a presnejšími, ale potenciálne menej súkromne chrániacimi, serverovými prístupmi.

Aby práca dosiahla svoj cieľ, snaží sa splniť nasledujúce implementačné ciele:

- **Vytvoriť rozšírenie Chrome Manifest V3 pre analýzu toxicity.**

Bude rozdelené do osvedčených postupov MV3, vrátane skriptov pre prístup k DOM, servisného pracovníka pre spúšťanie úloh na pozadí pre komunikáciu s modelmi alebo API a komponentov používateľského rozhrania pre vykresľovanie výsledkov používateľovi prostredníctvom kontextového okna alebo bočného panela.

- **Poskytnúť plne lokálnu inferenciu pomocou viacnásobného modelu toxicity.**

V tomto nastavení by sa predtrénovaný klasifikátor pre predikciu toxicity, ktorý je súčasťou rozšírenia (napr. graf TensorFlow.js), spustil výlučne na strane klienta. Systém by mal byť schopný prijať vybraný text, vykonať tokenizáciu a generovať skóre pre označenia, ako aj celkovú predikciu bez toho, aby klient odosielať akékoľvek údaje.

- **Povoliť režim vzdialenej inferencie prostredníctvom zabezpečeného proxy API.**

Okrem lokálneho riešenia by rozšírenie umožnilo aj vyvolanie vzdialeného riešenia pre toxicitu prostredníctvom minimálneho backendového proxy. To by umožnilo zapuzdrenie poverení pre poskytovateľov, obmedzenie rýchlosti, ako aj prepínanie medzi riešeniami, ako je transformačné riešenie alebo produkčné riešenie, bez nutnosti akejkoľvek zmeny v klientovi.

- **Pridať používateľské rozhranie pre skóre a prísnosť pre každé označenie.**
„Rozšírenie by malo poskytovať súhrnný hlavný verdikt (pravdepodobnosť toxickosti vs. netoxickosti) spolu so skóre pre podrobné verdikty o toxickosti (napr. urážka, hrozba, nenávisť na základe identity). Malo by byť možné nastaviť prísnosť detekcie pomocou jednoduchého rozhrania (napr. globálny posuvník) preložením úprav interných prahov do zrozumiteľného kontextu.“

- **Zvážiť voliteľné rozšírenia používateľskej interakcie.**

Zadanie práce pripúšťalo aj voliteľné rozšírenia, napríklad spätnú väzbu používateľa, podporu viacerých jazykov alebo export a anotáciu textov. V rámci práce bola spätná väzba používateľa zvažovaná v skorom návrhu rozhrania, avšak nebola zahrnutá do finálnej implementácie, keďže hlavný dôraz bol kladený na detekciu toxicity, používateľské rozhranie, lokálnu a vzdialenú inferenciu a vyhodnotenie modelov.

Súbežne s implementáciou si práca stanovila nasledujúce ciele týkajúce sa hodnotenia a analýzy:

- **Vyhodnotiť výkonnosť použitých modelov podľa dostupného experimentálneho protokolu.**

Pri hodnotení modelov budú použité štandardné metriky viacštítkovej klasifikácie, najmä presnosť, úplnosť, F1-skóre pre jednotlivé štítky, makropriemerovaný F1, mikropriemerovaný F1 a Hammingova strata. Rozsah hodnotenia sa môže líšiť podľa konkrétneho režimu inferencie a použitého modelu, keďže vzdialený model, pôvodný lokálny TF.js model a optimalizovaný lokálny model majú rozdielne technické vlastnosti a experimentálne obmedzenia.

- **Porovnať lokálny a vzdialený režim inferencie z hľadiska presnosti a praktickej použiteľnosti.**

Práca porovná správanie lokálneho a vzdialeného režimu na základe dostupných experimentálnych meraní. Vzdialený model bude hodnotený najmä z hľadiska klasifikačnej kvality, vplyvu nerovnováhy tried a nastavenia rozhodovacích prahov. Lokálny režim bude hodnotený ako súkromie zachovávajúci variant z hľadiska použiteľnosti v prehliadači, odozvy a kvality detekcie pri použitých lokálnych modeloch. Keďže jednotlivé modely majú rozdielne technické vlastnosti a obmedzenia, výsledky budú interpretované s ohľadom na konkrétny experimentálny protokol.

- **Analyzovať nastavenia kalibrácie a prahov z pohľadu používateľa.**

Práca preskúma, ako dobre skóre modelu zodpovedá pozorovaným mieram chybovosti a preskúma konfigurácie prahov podľa označenia, ktoré odrážajú rôzne ciele produktu, ako napríklad konzervatívnejšiu voľbu pre nenávisť založenú na identite ako pre generickú toxicitu. Výsledné nastavenia budú zdôvodnené numericky aj z hľadiska ich dôsledkov pre používateľskú skúsenosť.

- **Diskutovať vybrané aspekty spravodlivosti a skreslenia relevantné pre nasadenie.**

Práca sa bude venovať rizikám neúmyselného skreslenia najmä na analytickej úrovni a pri interpretácii výsledkov modelov. Cieľom nie je vykonať úplný audit spravodlivosti, ale pomenovať riziká, ktoré môžu ovplyvniť praktické nasadenie systému, najmä pri menšinových kategóriách, jazykovo odlišných textoch a rôznych používateľských kontextoch.

- **Zdokumentovať architektonické kompromisy a formulovať odporúčania pre odborníkov z praxe.**

Na základe skúseností z implementácie a experimentálnych výsledkov táto práca zhrnie hlavné kompromisy medzi lokálnou a vzdialenou inferenciou – súkromie, výkon, udržiavateľnosť, kvalita modelu – a načrtne pokyny pre vývojárov, ktorí sú ochotní integrovať detekciu toxicity do rozšírení prehliadača.

4 Praktická časť

Praktická časť práce opisuje návrh, implementáciu a overenie rozšírenia *Toxic Text Detector*. Nadväzuje na analytickú časť, v ktorej boli predstavené teoretické východiská, existujúce prístupy k detekcii toxicity a technické požiadavky na riešenie v prostredí webového prehliadača. V tejto kapitole sú tieto poznatky pretavené do konkrétneho návrhu architektúry, používateľského rozhrania a implementácie jednotlivých komponentov systému.

4.1 Úvod do praktickej časti

Na základe poznatkov z analytickej časti a formulovaných cieľov práce bolo navrhnuté a implementované rozšírenie prehliadača na detekciu toxického textu. Praktická časť práce opisuje celý proces tvorby riešenia: od požiadaviek, návrhu architektúry a používateľského rozhrania až po implementáciu jednotlivých komponentov, lokálnu inferenciu, vzdialený režim cez API a návrh testovania.

Navrhované riešenie vychádza z požiadavky vytvoriť rozšírenie prehliadača založené na Chrome Manifest V3, ktoré dokáže analyzovať text priamo na webovej stránke, poskytnúť používateľovi zrozumiteľný výsledok klasifikácie a zároveň rešpektovať požiadavky na súkromie, bezpečnosť a použiteľnosť. Riešenie preto kombinuje lokálne spracovanie textu, možnosť vzdialenej inferencie a používateľské rozhranie umožňujúce jednoduchú konfiguráciu správania doplnku.

Cieľom tejto kapitoly je ukázať, ako boli teoretické poznatky pretavené do konkrétneho praktického riešenia. Kapitola najskôr definuje požiadavky a návrh architektúry, následne opisuje návrh a prototypovanie používateľského rozhrania a v ďalších častiach sa venuje samotnej implementácii rozšírenia *Toxic Text Detector*.

4.2 Požiadavky na riešenie

Pred návrhom architektúry bolo potrebné určiť základné funkčné a nefunkčné požiadavky systému. Tieto požiadavky vychádzajú zo zadania práce, z analytickej časti a z očakávaného spôsobu používania rozšírenia pri prehliadaní webových stránok a online diskusií.

4.2.1 Funkčné požiadavky

Navrhované riešenie musí umožniť:

- analyzovať text vybraný používateľom na podporovanej webovej stránke,
- automaticky identifikovať textové úseky, ktoré môžu byť toxické, ak je zapnutá funkcia automatickej ochrany,
- zobrazíť výsledok klasifikácie vo forme hlavného verdiktu a skóre pre jednotlivé kategórie toxicity,
- prepínať medzi lokálnym a vzdialeným režimom inferencie,
- umožniť používateľovi meniť prísnosť detekcie a aktivovať alebo deaktivovať vybrané kategórie toxicity,
- uchovávať používateľské nastavenia medzi reláciami prehliadania,
- skryť a opätovne odhaliť označený obsah bez trvalého zásahu do štruktúry stránky.

4.2.2 Nefunkčné požiadavky

Okrem samotnej funkčnosti musí systém spĺňať aj nefunkčné požiadavky:

- **Súkromie:** systém má prednostne spracúvať text lokálne a odosielať ho mimo zariadenia len v prípade povoleného vzdialeného režimu.
- **Bezpečnosť:** rozšírenie musí rešpektovať pravidlá Chrome Manifest V3 a nesmie ukladať citlivé API kľúče v klientskom kóde.
- **Odozva:** klasifikácia má prebiehať tak, aby výrazne nenarúšala čítanie stránky.
- **Zrozumiteľnosť:** používateľské rozhranie má jasne zobrazovať, čo bolo detegované a aký typ toxicity bol vyhodnotený.

- **Rozšíriteľnosť:** architektúra má umožniť neskoršie doplnenie ďalších modelov, kategórií alebo jazykových režimov.

4.3 Návrh riešenia

Navrhované riešenie je koncipované ako ochranná vrstva medzi používateľom a webovým obsahom. Jeho cieľom nie je nahradiť moderovanie platforiem, ale poskytnúť používateľovi nástroj, ktorý mu pomôže identifikovať potenciálne škodlivý obsah a rozhodnúť sa, či ho chce zobraziť.

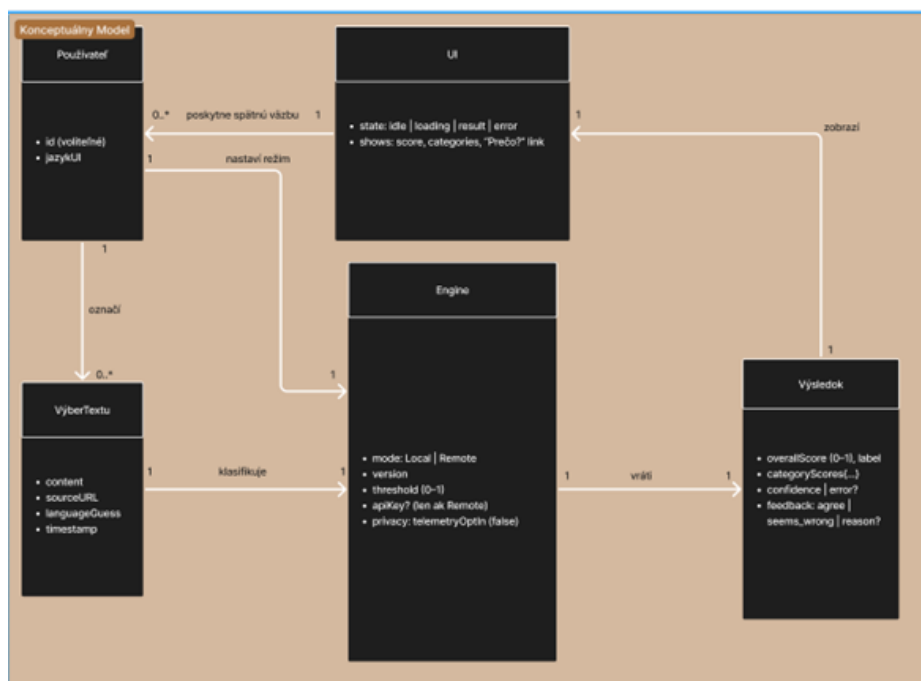
Z konceptuálneho hľadiska systém pozostáva zo štyroch hlavných vrstiev:

1. **Vrstva interakcie s používateľom**, ktorá zabezpečuje zobrazenie stavu ochrany, výsledkov analýzy a konfiguračných možností.
2. **Vrstva spracovania webového obsahu**, ktorá pracuje s DOM štruktúrou stránky, identifikuje relevantné textové úseky a pripravuje ich na analýzu.
3. **Vrstva inferencie**, ktorá vykonáva klasifikáciu textu lokálne alebo prostredníctvom vzdialeného API.
4. **Vrstva konfigurácie a perzistencie**, ktorá uchováva používateľské nastavenia, aktívne kategórie a stav rozšírenia.

Takéto rozdelenie oddeľuje zodpovednosti jednotlivých častí systému. Používateľské rozhranie sa stará o interakciu, obsahový skript o prácu so stránkou, inferenčná vrstva o vyhodnotenie textu a konfiguračná vrstva o stabilné uchovanie nastavení. Výhodou tohto prístupu je jednoduchšia údržba a možnosť meniť jednotlivé časti systému nezávisle od ostatných.

Vzťahy medzi používateľom, rozhraním, detekčným modulom a výsledkom klasifikácie sú znázornené na obrázku 4.1. Model ukazuje, že používateľ nekomunikuje priamo s detekčným modelom, ale cez rozhranie rozšírenia. UI vrstva prijíma používateľské akcie, zobrazuje výsledok klasifikácie a sprostredkúva konfiguráciu správania rozšírenia. Detekčný modul spracúva text podľa aktuálnych nastavení a vracia jednotný výsledok obsahujúci celkový verdikt a skóre jednotlivých kategórií.

Tento konceptuálny model bol použitý ako základ pre návrh architektúry, používateľského rozhrania a toku dát medzi jednotlivými časťami rozšírenia.



Obrázok 4.1: Konceptuálny model rozšírenia.

4.3.1 Architektúra rozšírenia

Architektúra riešenia vychádza z modelu rozšírení Chrome Manifest V3. Tento model prirodzene rozdeľuje funkcionality medzi content script, service worker a používateľské rozhranie rozšírenia. Každá z týchto častí má jasne definovanú úlohu.

Content script

Content script je komponent vložený do webovej stránky. V navrhovanom riešení zabezpečuje najmä:

- získanie textu vybraného používateľom,
- výber textových blokov vhodných na automatickú analýzu,
- odoslanie textu na klasifikáciu,
- prijatie výsledkov inferencie,
- vizuálne označenie alebo rozmazanie toxického obsahu.

Content script musí byť navrhnutý tak, aby zbytočne nezaťažoval stránku. Pri automatickom skenovaní preto nemá analyzovať celý DOM bez obmedzení, ale iba vhodné textové prvky spĺňajúce dĺžkové a viditeľnostné kritériá.

Service worker

Service worker predstavuje koordinačný prvok rozšírenia. V prostredí Manifest V3 nahrádza pôvodné trvalé background pages a pracuje udalostne. V návrhu zabezpečuje:

- prijímanie požiadaviek od používateľského rozhrania a content scriptu,
- rozhodovanie o použití lokálneho alebo vzdialeného režimu,
- sprostredkovanie komunikácie medzi komponentmi,
- vyvolanie opätovného skenovania stránky po zmene nastavení,
- správu akcií dostupných cez kontextové menu rozšírenia.

Keďže service worker nemá priamy prístup k DOM stránke, všetky operácie nad webovým obsahom musia byť delegované na content script. Tým sa zachováva bezpečnostný model rozšírenia a zároveň sa jasne oddeľuje práca so stránkou od globálnej logiky doplnku.

Používateľské rozhranie

Používateľské rozhranie je navrhnuté ako kombinácia rýchleho popup okna, obrazovky nastavení a prekrytia priamo na stránke. Popup slúži na rýchle zobrazenie stavu ochrany, počtu detegovaných prvkov a základných akcií. Obrazovka nastavení poskytuje priestor na podrobnejšiu konfiguráciu, napríklad výber aktívnych kategórií, prísnosti detekcie, režimu inferencie alebo whitelistu domén. Prekrytie na stránke umožňuje pracovať priamo s označeným obsahom.

4.3.2 Návrh režimov inferencie

Jedným z hlavných návrhových rozhodnutí je podpora dvoch režimov inferencie: lokálneho a vzdialeného. Tento hybridný prístup umožňuje vyvážiť požiadavky na súkromie, kvalitu klasifikácie a technickú realizovateľnosť.

Lokálny režim

V lokálnom režime prebieha klasifikácia priamo v prehliadači používateľa. Model je súčasťou rozšírenia alebo je načítaný ako jeho lokálny artefakt. Text preto nie je potrebné odosielať na externý server.

Výhodou lokálneho režimu je vyššia úroveň súkromia, nezávislosť od vzdialenej služby a rýchla odozva pri použití optimalizovaného modelu priamo v prehliadači. Nevýhodou je nižšia klasifikačná presnosť v porovnaní s výkonnejším vzdialeným modelom, najmä pri menej zastúpených kategóriách.

Vzdialený režim

Vo vzdialenom režime rozšírenie odošle text na vlastný backend alebo proxy API, ktoré zabezpečí klasifikáciu pomocou vzdialeného modelu. Tento režim je vhodný v prípadoch, keď je potrebná vyššia presnosť alebo jednoduchšie experimentovanie s rôznymi modelmi bez nutnosti meniť klientsku časť rozšírenia. Rýchlosť vzdialeného režimu závisí od konkrétneho nasadenia backendu, použitého hardvéru, veľkosti modelu a sieťovej latencie. V testovanom prostredí bolo API spustené lokálne na výkonnom notebooku, preto bola odozva vzdialeného režimu blízka lokálnemu režimu.

Proxy vrstva zároveň chráni citlivé údaje, pretože API kľúče nie sú uložené v klientskom kóde rozšírenia. Okrem toho môže zabezpečiť jednotný formát odpovede, obmedzenie dĺžky vstupu, limitovanie požiadaviek a meranie latencie.

Prepínanie medzi režimami

Prepínanie medzi režimami môže byť realizované manuálne používateľom v nastaveniach alebo podmienené podľa jazyka a dostupnosti lokálneho modelu. Z návrhového hľadiska je vhodné, aby bol lokálny režim predvolený a vzdialený režim sa použil len vtedy, keď ho používateľ povolí alebo keď je potrebný pre konkrétny scenár.

Takýto prístup podporuje zásadu *privacy-by-default*, pretože text sa mimo zariadenia neposiela automaticky bez jasnej potreby alebo nastavenia používateľom.

4.3.3 Návrh toku dát

Spracovanie textu v rozšírení prebieha ako postupnosť krokov medzi používateľským rozhraním, content scriptom, service workerom a inferenčnou vrstvou.

Typický scenár manuálnej kontroly vybraného textu pozostáva z nasledujúcich krokov:

1. Používateľ označí text na webovej stránke.
2. Content script získa označený text a pripraví požiadavku na analýzu.

3. Požiadavka sa odošle príslušnej časti rozšírenia podľa zvoleného režimu.
4. Inferenčná vrstva vykoná klasifikáciu textu.
5. Výsledok sa vráti späť content scriptu.
6. Content script zobrazí výsledok alebo vizuálne označí analyzovaný text.

Pri automatickom skenovaní stránky je proces podobný, ale namiesto jedného vybraného textu sa pracuje s množinou textových úsekov. Aby sa znížila záťaž stránky, návrh počíta s obmedzením počtu analyzovaných prvkov, dávkovým spracovaním, cache výsledkov a filtrovaním nerelevantných DOM prvkov.

4.3.4 Návrh práce s kategóriami toxicity a prahmi

Keďže detekcia toxicity je problém viacnásobnej klasifikácie, systém nemá poskytovať iba jedno binárne rozhodnutie. Každý analyzovaný text môže patriť do viacerých kategórií súčasne, napríklad všeobecná toxicita, urážka, obscénnosť, hrozba alebo nenávisť voči identite.

Výsledok klasifikácie by preto mal obsahovať:

- hlavný verdikt, či ide o potenciálne toxický obsah,
- skóre pre jednotlivé kategórie,
- informáciu o tom, ktoré kategórie prekročili nastavené prahy,
- voliteľné upozornenie pri nižšej spoľahlivosti výsledku.

Prahy detekcie nemusia byť rovnaké pre všetky kategórie. Napríklad hrozby môžu vyžadovať citlivejší prah než všeobecná toxicita. Preto návrh počíta s internými prahmi pre jednotlivé kategórie a s používateľským nastavením prísnosti, ktoré tieto prahy upravuje jednoduchým a zrozumiteľným spôsobom.

4.3.5 Bezpečnostné a súkromnostné opatrenia

Bezpečnosť a ochrana súkromia patria medzi hlavné kritériá návrhu. Systém preto uplatňuje tieto zásady:

- lokálne spracovanie je preferovaný režim,
- vzdialené spracovanie je používateľovi zreteľne signalizované,
- API kľúče sa neukladajú v klientskom kóde,

- rozšírenie nepoužíva vzdialene hostovaný vykonateľný kód,
- analyzujú sa iba relevantné textové časti stránky,
- uchováva sa len nevyhnutné minimum nastavení a spätnej väzby.

Takýto návrh je v súlade s technickými obmedzeniami Manifest V3 a zároveň podporuje princípy minimalizácie údajov a transparentnosti voči používateľovi.

4.4 Prototypovanie používateľského rozhrania

Používateľské rozhranie doplnku *Toxic Text Detector* bolo navrhované iteratívne formou prototypovania v dvoch hlavných iteráciách. Cieľom rozhrania bolo umožniť používateľovi rýchlo rozpoznať potenciálne toxické časti online diskusií, pričom zásah do čítania mal zostať čo najmenší. Dôležitou požiadavkou bola aj reverzibilita akcií, pretože používateľ musí mať možnosť označený text kedykoľvek zobrazíť alebo opätovne skryť.

Prvá iterácia vznikla ako interaktívny prototyp vo Figma. Slúžila najmä na overenie rozloženia obrazoviek, pomenovania prvkov a základných interakčných vzorov, ako je rozmazanie toxického textu, odhalenie obsahu, otvorenie vysvetlenia a prístup k nastaveniam. Druhá iterácia bola vytvorená ako bežiaci HTML/React prototyp pomocou technológií Vite a React. Táto verzia už lepšie približovala správanie finálneho rozšírenia, najmä popup okno, stránku nastavení a prekrytie priamo na webovej stránke.

4.4.1 UX výskum

UX výskum sa zameriaval na situáciu, v ktorej používateľ prehliada verejné diskusie alebo dlhšie vlákna komentárov. V takomto prostredí je dôležité, aby ochrana nevyžadovala zložité rozhodovanie pri každom komentári. Používateľ má byť chránený pred nechceným obsahom, ale zároveň má mať možnosť zobrazíť si text v prípade potreby, napríklad pri moderovaní alebo pri posudzovaní kontextu diskusie.

Z návrhového hľadiska boli sledované najmä tieto otázky:

- Ako môže rozhranie signalizovať toxicitu bez zbytočného prerušenia čítania?
- Ako zachovať akcie reverzibilné, aby používateľ nestratil kontrolu nad obsahom?

- Aké minimum vysvetlenia je potrebné, aby používateľ výsledku detekcie rozumel?
- Ako sprístupniť nastavenia tak, aby boli dostupné, ale nie rušivé?

Na základe analýzy potrieb boli identifikované dve typické skupiny používateľov:

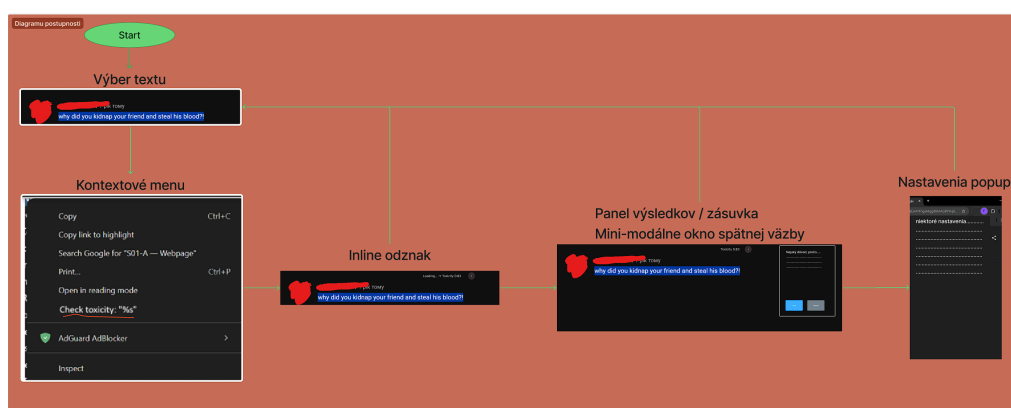
- **Bežný používateľ:** chce rýchlo prehliadať diskusie bez zbytočných zásahov do čítania.
- **Moderátor alebo správca komunity:** potrebuje obsah zobrazíť aj v prípade ochrany, no zároveň chce jasné označenie rizikových častí.

4.4.2 Návrh sekvencií obrazoviek

Návrh toku obrazoviek bol postavený okolo kontinuálnej slučky prehliadania: *čítať* → *detegovať* → *voliteľne odhaliť* → *pochopiť dôvod* → *upraviť nastavenia* → *pokračovať v čítaní*.

Interakcia je ukotvená priamo v zozname komentárov, zatiaľ čo globálna konfigurácia je dostupná cez popup rozšírenia a obrazovku nastavení. Na zníženie interakčných nákladov používa UI rovnaký štrukturálny vzor naprieč obrazovkami: jasnú hlavičku, jednu primárnu obsahovú oblasť a obmedzenú množinu akcií.

Základné obrazovky a akcie sú znázornené na obrázku 4.2. Tento návrh bol použitý ako podklad pre implementáciu bežiaceho HTML/React prototypu.



Obrázok 4.2: Návrh postupností obrazoviek a navigácie rozšírenia.

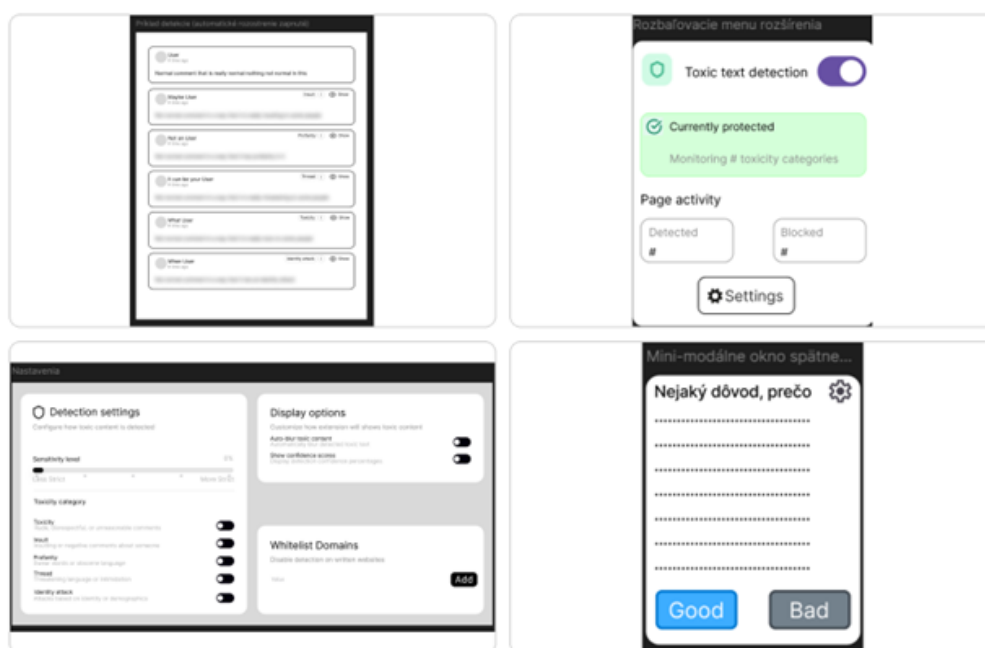
Navrhnuté boli najmä tieto časti používateľského rozhrania:

- **Prekrytie na stránke:** toxické komentáre sú predvolene rozmazané, ak je zapnuté automatické rozmazanie.

- **Ovládanie na úrovni komentára:** používateľ môže text zobrazíť alebo skryť a otvoriť vysvetlenie dôvodu označenia.
- **Popup rozšírenia:** zobrazuje stav ochrany, počet aktívnych kategórií a základné metriky aktivity.
- **Nastavenia:** umožňujú zapnúť alebo vypnúť kategórie, upraviť prísnosť detekcie a spravovať whitelist domén.

4.4.3 Prototypy používateľského rozhrania

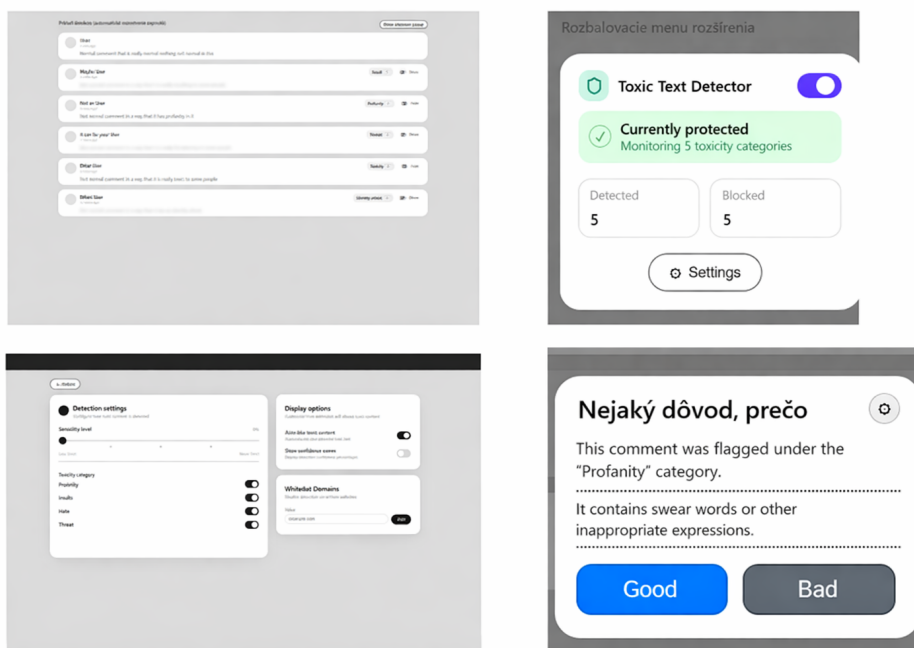
Prvý prototyp bol vytvorený vo Figma ako interaktívny mock-up so strednou vernosťou. Jeho cieľom bolo overiť mentálny model a základný interakčný vzor bez implementačných detailov, najmä rozmazanie toxického textu, zobrazenie a skrytie na úrovni komentára, prístup k nastaveniam a jednoduché mini-modálne okno spätnej väzby. Mini-modálne okno spätnej väzby zostalo iba súčasťou skorého návrhu používateľského rozhrania. Do finálnej implementácie rozšírenia nebolo zahrnuté, pretože prvé riešenie nepôsobilo dostatočne použiteľne a jeho dopracovanie by si vyžadovalo samostatný návrh, ukladanie dát a ďalšie testovanie.



Obrázok 4.3: Prvý prototyp vo Figma.

Druhý prototyp implementoval overený interakčný vzor v bežiacom HTML/React prototype. Táto iterácia zvýšila fidelitu návrhu a zabezpečila, že všetky prepínače a stavy sa správajú konzistentne: globálne zapnutie ochrany, automatické rozmazanie, zapnutie a vypnutie kategórií a voliteľné zobrazenie confidence

skóre. Stav bol centralizovaný cez kontext nastavení, aby popup, obrazovka nastavení a karty komentárov na stránke vždy reflektovali rovnakú konfiguráciu.



Obrázok 4.4: Ukážka implementovaného rozhrania v druhom prototypu.

Pre potreby dokumentácie sú uvedené aj odkazy na zdroje návrhu:

- Zdrojový súbor prototypu: [Figma Design File](#).
- Interaktívny náhľad prototypu: [Figma Prototype](#).
- Detailný popis, testovacie scenáre a výsledky: [FigJam – Prototypovanie a testovanie](#).

4.4.4 Zmeny na základe prototypovania

Na základe priebežných zistení počas návrhu boli realizované tieto kľúčové zmeny:

- **Zlepšenie vizuálnej hierarchie v kartách komentárov:** zvýraznenie hlavičky a zníženie vizuálnej dominance sekundárnych informácií.
- **Presun kontextových akcií do hlavičky karty:** rýchlejšia objaviteľnosť ovládania, napríklad reveal/hide a vysvetlenia dôvodu označenia.
- **Funkčné prepínače kategórií:** vypnutie kategórie okamžite upraví správanie detekcie a rozmazania pre daný typ toxicity.

- **Zjednotenie stavov v celom UI:** popup, nastavenia a prekrytie na stránke vždy zobrazujú tú istú konfiguráciu.
- **Jasnejší systém stavov:** chránený stav využíva výraznejšiu status kartu, zatiaľ čo vypnutý stav používa neutrálnejší vizuál.

Kompletný zdrojový kód prototypu je dostupný v repozitári:

- **GIT projekt:** git.kpi.fei.tuke.sk.

4.5 Implementácia rozšírenia

Výsledná implementácia rozšírenia *Toxic Text Detector* pozostáva zo štyroch hlavných častí: konfiguračnej vrstvy rozšírenia, používateľského rozhrania vytvoreného v Reacte, background logiky vo forme service workera a content scriptu, ktorý vykonáva lokálnu detekciu toxicity priamo na stránke.

Konfigurácia rozšírenia je definovaná v súbore *manifest.json*. Použitý je formát Chrome Manifest V3, v ktorom je ako service worker zaregistrovaný súbor *background.js*. Manifest zároveň určuje popup okno *popup.html*, stránku nastavení *options.html*, content scripty a web-accessible resources potrebné pre TensorFlow.js a WASM backend.

Používateľské rozhranie rozšírenia je implementované v priečinku *src*. Popup rozhranie sa inicializuje cez súbor *popupMain.jsx*, stránka nastavení cez *optionsMain.jsx*. Obe časti používajú spoločný kontext *SettingsContext.jsx*, ktorý zabezpečuje jednotnú prácu s nastaveniami a ich uloženie do synchronizovaného úložiska prehliadača.

Samotná lokálna detekcia toxicity je implementovaná v súbore *contentScript.js*. Tento súbor obsahuje načítanie modelu, výber kandidátnych textových uzlov, klasifikáciu textu, prácu s cache a vizuálne označovanie toxického obsahu na stránke. Background skript *background.js* vytvára kontextové menu a sprostredkúva komunikáciu medzi popup rozhraním a content scriptom aktívnej karty.

4.5.1 Build proces a konfigurácia

Build proces bol realizovaný pomocou nástroja Vite. Konfigurácia v súbore *vite.config.js* definuje dva vstupné body: *popup.html* a *options.html*. Pri produkčnom zostavení sa tak generujú dve samostatné HTML stránky rozšírenia, pričom každá načítava svoj vlastný React vstupný bod.

Súbor *popup.html* načítava skript *popupMain.jsx*, zatiaľ čo *options.html* načítava súbor *optionsMain.jsx*. Popup rozhranie a stránka nastavení sa preto zostavujú nezávisle, ale stále používajú spoločné komponenty a zdieľaný stav.

Závislosti projektu sú definované v súbore *package.json*. Pre používateľské rozhranie sú použité knižnice React a React DOM. Lokálna inferencia využíva TensorFlow.js, model Toxicity a WASM backend. Produkčný build sa vytvára príkazom *npm run build*.

Manifest deklaruje používanie Manifest V3, service worker *background.js*, popup okno rozšírenia, stránku nastavení, content scripty a web-accessible resources. V rámci content scriptov sa načítavajú knižnice pre jadro TensorFlow.js, konverziu modelu, backendy WASM a CPU, model *toxicity.min.js*, pomocný kontrakt *toxicityContract.global.js* a hlavná implementácia *contentScript.js*. Web-accessible resources obsahujú najmä WASM súbory a priečinok s kernelmi, ktoré TensorFlow.js potrebuje pri lokálnom behu modelu.

4.5.2 Implementácia popup rozhrania

Popup rozhranie je inicializované v súbore *popupMain.jsx*. Tento súbor vytvára React koreň, obalí popup komponent do *SettingsProvider* a pomocou hooku *useTabStats* priebežne načítava štatistiky z aktívnej karty.

Komponent *PopupRoot* získava zo stavového kontextu informáciu o tom, či je rozšírenie zapnuté, a zároveň z hooku *useTabStats* načítava počty detegovaných a zablokovaných prvkov. Po kliknutí na prepínač ochrany sa zavolá funkcia *toggleEnabled()* a následne sa background skriptu odošle správa pre opätovné skenovanie aktívnej karty. Tým sa vyžiada nové skenovanie aktuálnej stránky.

Samotný vizuál popup okna je implementovaný v komponente *ExtensionPopup.jsx*. Tento komponent prijíma všetky potrebné dáta cez props: stav ochrany, callback na jej prepnutie, callback na otvorenie nastavení, počet aktívnych kategórií a štatistiky z aktuálnej stránky. Popup zobrazuje hlavnú kartu so stavom ochrany, prepínač, dva informačné bloky s počtom detegovaných a blokovaných prvkov a tlačidlo pre otvorenie nastavení. Štýly popup okna sú definované v súbore *popup.css*.

4.5.3 Implementácia stránky nastavení

Stránka nastavení sa inicializuje v súbore *optionsMain.jsx*. Rovnako ako popup rozhranie používa *SettingsProvider*, takže všetky zmeny nastavení sú zapisované do rovnakého dátového modelu.

Hlavným komponentom tejto časti je *SettingsLayout.jsx*. Tento komponent používa hodnoty a funkcie zo súboru *SettingsContext.jsx* a vykresľuje ich vo forme viacerých kariet rozdelených podľa typu nastavení. Implementácia obsahuje posuvník citlivosti detekcie, prepínače jednotlivých kategórií toxicity, prepínač automatického rozmazania toxického textu, prepínač zobrazenia confidence skóre, formulár pre pridanie domény do whitelistu, tlačidlo na reset nastavení, export a import nastavení vo formáte JSON a tlačidlo na manuálne opätovné skenovanie aktívnej karty.

Pri zmene citlivosti alebo pri prepnutí niektorej kategórie komponent okamžite volá funkciu *pingRescan()*, ktorá odošle background skriptu správu pre opätovné skenovanie aktívnej karty. Takto sa zmena nastavenia okamžite prejaví aj na otvorenej stránke bez potreby ručného obnovenia karty. Štýly tejto časti sú definované v súbore *settings.css*.

4.5.4 Správa nastavení a perzistencia

Zdieľaný stav rozšírenia je implementovaný v súbore *SettingsContext.jsx*. Tento súbor definuje konštantu *DEFAULT_SETTINGS*, ktorá obsahuje predvolený stav ochrany, citlivosť, nastavenia zobrazenia, aktívne kategórie toxicity a whitelist domén.

Po inicializácii *SettingsProvider* sa z úložiska načíta uložený objekt nastavení. Ak nastavenia v úložisku neexistujú, použijú sa predvolené hodnoty. Po každej zmene stavu sa aktualizovaná konfigurácia uloží späť.

Kontext zároveň poskytuje funkcie *toggleEnabled()*, *setSensitivity()*, *toggleAutoBlur()*, *toggleShowConfidence()*, *toggleCategory()*, *addWhitelist()* a *removeWhitelist()*, ktoré používajú popup a stránka nastavení. Normalizácia domén je implementovaná pomocnou funkciou *normalizeDomain()*, ktorá odstraňuje protokol, cestu a prefix *www*. Táto funkcia sa používa pri ukladaní whitelistu, aby sa rôzne zápisy tej istej domény neukladali ako odlišné hodnoty.

4.5.5 Background logika a komunikácia

Service worker rozšírenia je implementovaný v súbore *background.js*. Táto časť nepracuje s DOM stromom stránky, ale zabezpečuje centrálné akcie rozšírenia a komunikáciu s content scriptom.

Pri inštalácii a štarte rozšírenia sa vytvárajú tri položky kontextového menu: kontrola vybraného textu na toxicitu, skenovanie stránky na toxicitu a odstránenie existujúcich označení toxicity. Po kliknutí na niektorú z týchto položiek service

worker vyhladá aktívnu kartu a odošle jej správu cez rozhranie kariet, čím spustí príslušnú akciu v content scripte.

Background skript zároveň spracúva správy od popup rozhrania. Pri požiadavke na získanie štatistík najprv zistí aktívnu kartu a potom od content scriptu vyžiada štatistiky stránky. Pri požiadavke na opätovné skenovanie aktívnej karty odošle content scriptu správu na nové skenovanie stránky. Takto popup rozhranie získa údaje o stránke bez priameho prístupu k DOM.

Keďže background skript v MV3 nemá priamy prístup k obsahu stránky, všetky akcie nad obsahom musia byť delegované na content script. Tento model sa používa pri kontrole vybraného textu, pri skenovaní celej stránky aj pri odstraňovaní existujúcich označení.

4.6 Implementácia lokálnej inferencie toxicity

Táto časť sa venuje lokálnemu spracovaniu textu priamo v prostredí prehliadača. Opisuje načítanie modelu, normalizáciu výstupu, klasifikáciu textových úsekov, prácu s cache a spôsob vizuálneho označovania toxického obsahu na webovej stránke. Lokálna inferencia je dôležitou súčasťou riešenia, pretože podporuje ochranu súkromia používateľa a znižuje závislosť od vzdialených služieb.

4.6.1 Načítanie modelu a normalizácia výstupu

Lokálny model je načítavaný v content scripte funkciou *loadToxicityModel()*. Implementácia používa premennú *modelPromise*, aby sa model nenačítaval opakovane pri každej klasifikácii. Po prvom zavolaní sa vytvorený príslub znovu používa pri ďalších požiadavkách.

Funkcia najprv nastaví cestu k WASM súboru pomocou rozhrania TensorFlow.js, pričom cesta sa generuje cez runtime API rozšírenia. Následne sa systém pokúsi aktivovať backend *wasm*. Ak inicializácia zlyhá, použije sa fallback na backend *cpu*.

Po úspešnej inicializácii backendu sa načíta model toxicity, pričom zoznam štítkov obsahuje kategórie *toxicity*, *severe toxicity*, *obscene*, *insult*, *threat* a *identity attack*. Prahová hodnota modelu je pri načítaní nastavená na nulovú hodnotu, pretože konečné rozhodovanie o toxicite sa nerobí priamo na úrovni predvoleného prahu modelu, ale cez vlastnú logiku implementovanú v súbore *toxicityContract.global.js*.

Súbor *toxicityContract.global.js* implementuje pomocnú vrstvu medzi surovým výstupom modelu a logikou rozšírenia. Definuje zoznam štítkov, základné pra-

hové hodnoty, funkcie pre normalizáciu požiadavky, normalizáciu skóre, úpravu prahov podľa používateľskej prísnosti a výpočet finálneho verdiktu. Najdôležitejšou časťou je funkcia, ktorá z prijatých skóre vytvorí jednotný objekt odpovede obsahujúci režim detekcie, názov modelu, normalizované skóre, aplikované prahy, výsledný verdikt, spustené kategórie a latenciu klasifikácie.

Používateľské rozhranie pracuje s kategóriami *Toxicity*, *Insult*, *Profanity*, *Threat* a *Identity attack*. Tieto kategórie sa v content scripte mapujú na konkrétne modelové štítky pomocou funkcie *buildContractSettingsFromUI()*. Rovnako sa tu prevádza používateľská hodnota *sensitivity* na interný parameter *strictness*, ktorý sa následne použije pri úprave prahov.

4.6.2 Klasifikácia textu a práca s cache

Hlavná logika klasifikácie je implementovaná v súbore *contentScript.js*. Súbor definuje limity pre minimálnu a maximálnu dĺžku textu, maximálny počet kandidátnych textových uzlov, veľkosť dávky pri klasifikácii a veľkosť LRU cache.

Pri manuálnej kontrole vybraného textu sa používa funkcia *handleCheckSelection()*. Tá najprv načíta aktuálne nastavenia, overí, či je rozšírenie zapnuté a či doména nie je vo whiteliste. Potom získa text aktuálneho výberu pomocou funkcie *getSelectedText()*. Ak text spĺňa dĺžkové obmedzenia, odošle sa na klasifikáciu cez funkciu *classifySingleText()*.

Pri plnom skenovaní stránky sa najprv cez funkciu *collectPageCandidateTextNodes()* vyberú vhodné textové uzly z DOM stromu. Do spracovania sa nezaraďujú prvky ako *script*, *style*, *input*, *textarea* alebo už označené časti textu. Zohľadňuje sa aj minimálna dĺžka textu a viditeľnosť prvku vo viewport priestore. Ak stránka prekročí definovaný limit počtu kandidátov alebo objemu textu, používateľ dostane upozornenie, že plný lokálny scan nie je vhodný a má použiť kontrolu len vybraného textu.

Samotná dávková klasifikácia je implementovaná vo funkcii *classifyTextNodes()*. Texty sa najprv normalizujú, potom sa porovnávajú s cache a unikátne texty sa rozdelia do dávok s definovanou veľkosťou. Na každú dávku sa následne zavolá klasifikačná funkcia, ktorá spracuje výstup modelu a vráti jednotnú odpoveď podľa interného kontraktu rozšírenia.

Na obmedzenie opakovaných výpočtov bola implementovaná vlastná LRU cache cez funkciu *createLRU()*. Cache uchováva výsledky klasifikácie pre konkrétny text a konkrétnu kombináciu aktívnych štítkov a prísnosti. Pri zmene nastavení v synchronizovanom úložisku sa cache automaticky vymaže, aby sa staré výsledky nepoužívali po zmene citlivosti alebo aktívnych kategórií.

4.6.3 Vizuálne označenie toxického obsahu

Po detekcii toxického textu content script upravuje DOM stránky priamo. Funkcia *createMarkerFromTextNode()* nahradí pôvodný textový uzol vlastným prvkom *span* s interným atribútom rozšírenia. Vo vnútri tohto elementu sa vytvorí textový prvok, blur efekt a toolbar alebo inline chip s kategóriou a skóre.

Ak je v nastaveniach aktívne automatické rozmazanie, text dostane príslušnú blur triedu. Používateľ môže kliknutím na tento text blur efekt zapnúť alebo vypnúť. Tým sa zachováva reverzibilita interakcie bez potreby opätovného načítania stránky.

Tooltip a informačný chip sa vytvárajú pomocou funkcie *createToolbarElement()*, ktorá zoberie primárnu kategóriu výsledku a confidence skóre. Štýly všetkých týchto prvkov sa vkladajú dynamicky cez funkciu *injectStyles()*, ktorá do dokumentu pridáva vlastný štýlový prvok. Odstránenie existujúcich značiek toxicity zabezpečuje funkcia *clearTTDDDecorations()*, ktorá prejde všetky aktívne označené prvky, odstráni pomocné toolbar prvky a obnoví pôvodný text stránky.

4.7 Vzdialený režim inferencie cez API

Popri lokálnom režime je v projekte pripravený aj vzdialený režim inferencie, určený pre scenáre, kde je výhodnejšie použiť výkonnejší model mimo prehliadača. Ide napríklad o viacjazyčné spracovanie, experimenty s iným modelom alebo porovnanie presnosti a latencie.

Rozšírenie v tomto režime neposiela požiadavky priamo na externého poskytovateľa, ale komunikuje s vlastným minimálnym backendom, teda proxy API. Tento backend poskytuje jednotné rozhranie */predict*, autorizáciu cez Bearer token, obmedzenia vstupu, napríklad maximálnu dĺžku textu, a meranie latencie.

Výstup API má stabilnú štruktúru pozostávajúcu z per-label skóre, výsledného verdiktu a meta-informácií. Vďaka tomu môže používateľské rozhranie pracovať s rovnakým formátom dát bez ohľadu na to, či výsledok pochádza z lokálneho alebo vzdialeného režimu. Proxy vrstva zároveň chráni API kľúče, keďže sa nenachádzajú v klientskom kóde rozšírenia.

4.8 Doplnujúce mechanizmy implementácie

Okrem hlavnej logiky detekcie toxicity obsahuje rozšírenie aj viacero doplnkových mechanizmov, ktoré zlepšujú použiteľnosť, kontrolu používateľa a prehľad

o stave systému. Medzi tieto mechanizmy patrí whitelist domén a získavanie štatistík z aktívnej stránky.

4.8.1 Whitelist domén

Whitelist domén je spravovaný v používateľskom rozhraní cez komponent *SettingsLayout.jsx*. Domény možno pridávať cez vstupné pole a odstraňovať cez tlačidlo pri každej položke zoznamu.

V content scripte sa whitelist používa cez funkciu *domainIsWhitelisted()*. Pri otvorení stránky alebo pri zmene nastavení content script porovná aktuálny host s normalizovaným zoznamom domén. Ak je doména vo whiteliste, detekcia sa nevykoná a existujúce označenia sa odstránia cez funkciu *clearTTDDecorations()*.

4.8.2 Získavanie štatistík z aktívnej stránky

Na zobrazovanie údajov v popup okne je implementovaný hook *useTabStats.js*. Tento hook periodicky posiela background skriptu požiadavku na získanie štatistík. Background následne vyžiada od content scriptu aktuálne štatistiky stránky.

Content script vracia údaje cez funkciu *getCurrentStats()*, ktorá spočíta počet označených textových úsekov, počet momentálne rozmazaných úsekov, počet aktívnych kategórií, poslednú vykonanú akciu, poslednú stavovú správu a informáciu, či práve prebieha klasifikácia. Tieto údaje sa v popup okne zobrazujú ako activity metriky.

4.9 Praktické obmedzenia implementácie

Počas implementácie sa ukázalo, že lokálna detekcia toxicity v rozšírení prehliadača je realizovateľná, ale vyžaduje technické obmedzenia. Najmä pri väčších stránkach nie je vhodné analyzovať celý DOM bez filtrov a limitov. Preto boli pridané obmedzenia počtu kandidátnych textových uzlov, celkovej dĺžky textu a dĺžky manuálne vybraného úseku.

Ďalším praktickým obmedzením je prostredie Chrome Manifest V3. Service worker je udalostne riadený a nemusí zostať trvalo aktívny. Z tohto dôvodu bolo potrebné navrhnuť komunikáciu tak, aby dôležité operácie nad obsahom stránky vykonával content script a aby stav rozšírenia bol uchovávaný v perzistentnom úložisku.

Medzi obmedzenia finálnej implementácie patrí aj absencia plnohodnotného mechanizmu spätnej väzby používateľa pri nesprávnej klasifikácii. Tento prvok bol v zadaní uvedený ako voliteľné rozšírenie a bol zvažovaný v skorom návrhu používateľského rozhrania, avšak nebol zahrnutý do finálnej verzie rozšírenia. Dôvodom bolo, že prvý návrh spätnej väzby nepôsobil dostatočne použiteľne a jeho dopracovanie by si vyžadovalo ďalšie návrhové rozhodnutia, spôsob ukladania údajov, ochranu súkromia a samostatné testovanie.

4.10 Príprava experimentálneho overenia

Aby bolo možné overiť funkčnosť a kvalitu riešenia, praktická časť počíta s viacerými úrovňami testovania: funkčným testovaním, výkonnostným testovaním, testovaním používateľskej skúsenosti a hodnotením kvality klasifikácie.

Na overenie správania rozšírenia a použitých modelov boli pripravené pomocné testovacie skripty a experimentálne postupy. Ich cieľom nebolo tvoriť samostatnú funkcionálnu rozšírenia, ale umožniť opakovateľné meranie vlastností, ktoré boli definované v cieľoch práce a následne vyhodnotené v samostatnej kapitole.

Testovacie časti boli zamerané najmä na overenie správania lokálneho modelu, porovnanie backendov pri lokálnej inferencii, meranie klasifikačných metrík na validačných dátach a kontrolu vplyvu rozhodovacích prahov. Pri používateľskom rozhraní boli pripravené scenáre úloh, ktoré umožnili overiť zrozumiteľnosť interakcií, reverzibilitu rozmazania a použiteľnosť nastavení.

Výsledky týchto testov nie sú uvádzané v praktickej časti, pretože sú súčasťou kapitoly venovanej vyhodnoteniu riešenia.

4.11 Zhrnutie praktickej časti

Praktická časť predstavila návrh a implementáciu rozšírenia prehliadača *Toxic Text Detector*. Riešenie je založené na Chrome Manifest V3 a pozostáva z používateľského rozhrania, content scriptu, service workera, vrstvy perzistentných nastavení a inferenčného mechanizmu.

V kapitole boli najskôr definované funkčné a nefunkčné požiadavky, následne bol opísaný konceptuálny model a architektúra riešenia. Osobitná pozornosť bola venovaná hybridnému modelu inferencie, ktorý kombinuje lokálne spracovanie textu s možnosťou vzdialeného režimu cez API. Lokálny režim podporuje ochranu súkromia a pri optimalizovanom modeli poskytuje rýchlu odozvu pri-

mo v prehliadači, zatiaľ čo vzdialený režim vytvára priestor pre presnejšie modely a jednoduchšiu experimentálnu výmenu inferenčnej logiky.

Ďalšia časť kapitoly sa venovala prototypovaniu používateľského rozhrania. Boli opísané dve iterácie návrhu: Figma prototyp a následný HTML/React prototyp. Tieto prototypy pomohli overiť hlavné interakčné princípy, ako je rozmazanie toxického textu, možnosť odhalenia obsahu, zobrazenie dôvodu označenia a konfigurácia kategórií.

Implementačná časť opísala konkrétne komponenty rozšírenia, build proces, prácu s nastaveniami, background logiku, lokálnu inferenciu pomocou TensorFlow.js, cache výsledkov, vizuálne označovanie toxického obsahu a doplnkové mechanizmy, ako je whitelist domén a získavanie štatistík z aktívnej stránky. Zároveň boli pomenované praktické obmedzenia riešenia, najmä výkonové limity pri lokálnej analýze väčších stránok a jazykové obmedzenie použitého lokálneho modelu.

Navrhnuté a implementované riešenie tak vytvára základ pre overenie funkčnosti, používateľskej skúsenosti a kvality klasifikácie v ďalšej časti práce.

5 Vyhodnotenie riešenia

Táto kapitola sa venuje vyhodnoteniu navrhnutého a implementovaného riešenia *Toxic Text Detector*. Hodnotenie je rozdelené na viacero častí, ktoré spoločne posudzujú používateľské rozhranie, vzdialený režim inferencie, lokálny TF.js model a optimalizovaný lokálny model. Cieľom kapitoly je overiť, do akej miery riešenie spĺňa požiadavky definované v predchádzajúcich častiach práce, najmä z hľadiska použiteľnosti, presnosti, výkonu a ochrany súkromia.

5.1 Metodika vyhodnotenia

Vyhodnotenie navrhnutého riešenia bolo realizované na viacerých úrovniach, aby bolo možné posúdiť nielen technickú funkčnosť rozšírenia, ale aj kvalitu používateľského rozhrania a správanie modelov v lokálnom aj vzdialenom režime inferencie. Hodnotenie preto zahŕňa používateľské testovanie prototypov rozhrania, podrobnejšie experimenty so vzdialeným modelom toxicity, orientačné vyhodnotenie pôvodného lokálneho TF.js modelu a vyhodnotenie optimalizovaného lokálneho modelu vo formáte TensorFlow.js GraphModel.

Pri používateľskom rozhraní bolo cieľom overiť zrozumiteľnosť interakcií, objaviteľnosť hlavných ovládacích prvkov a celkové vnímanie použiteľnosti. Hodnotenie prebiehalo formou scenárových úloh, krátkeho post-test dotazníka s 5-bodovou Likertovou škálou, kvalitatívnych pozorovaní typu think-aloud a následného krátkeho rozhovoru.

Pri modelovej časti bolo cieľom vyhodnotiť kvalitu detekcie toxicity pomocou metrík viacštítkovej klasifikácie, pričom rozsah hodnotenia sa líšil podľa konkrétneho režimu a modelu. Pri vzdialenom režime boli podrobnejšie analyzované viaceré verzie modelu, nerovnováha tried a rozhodovacie prahy. Pri lokálnom režime bolo hodnotenie rozdelené na orientačné overenie pôvodného predtrénovaného TF.js modelu a následné vyhodnotenie optimalizovaného lokálneho modelu, ktorý bol použitý ako praktickejší variant pre finálne riešenie.

5.2 Vyhodnotenie používateľského rozhrania

Používateľské rozhranie bolo vyhodnocované v dvoch iteráciách, ktoré zodpovedajú postupnému vývoju prototypu. Prvá iterácia overovala základný mentálny model a hlavné interakčné vzory, zatiaľ čo druhá iterácia sa sústredila na konzistentnosť stavov, použiteľnosť ovládacích prvkov a realistickejšie správanie rozhrania. Vyhodnotenie tejto časti preto porovnáva zistenia z oboch prototypov a ukazuje, ako sa spätná väzba od účastníkov testovania premietla do ďalšieho návrhu.

5.2.1 Prvý prototyp a jeho verifikácia

Prvý prototyp bol vytvorený vo Figma ako interaktívny mock-up so strednou vernosťou. Jeho cieľom bolo overiť mentálny model a základný interakčný vzor bez implementačných detailov: rozmazanie toxického textu, zobrazenie a skrytie na úrovni komentára, prístup k nastaveniam a jednoduché mini-modálne okno spätnej väzby.

Metóda verifikácie. Testovanie prebiehalo formou krátkej moderovanej relácie. Účastníci riešili scenárové úlohy, napríklad zobrazíť a opäť skryť rozmazaný komentár, nájsť dôvod alebo kategóriu a zmeniť nastavenie, pričom priebežne verbalizovali svoje rozhodnutia formou think-aloud. Po úlohách vyplnili krátky post-test dotazník s 5-bodovým hodnotením vybraných častí UI, doplnený otvorenými otázkami.

Účastníci. Prvý prototyp bol testovaný s 5 účastníkmi ($n = 5$).

Výsledky a interpretácia.

- **Kvantitatívne:** Interakčný koncept bol hodnotený pozitívne a používanie bolo vnímané ako jednoduché. Najlepšie dopadlo rozbaľovacie menu rozšírenia, kde prevažovali hodnotenia 5/5 a ojedinele 4/5. Príklady detekcie a nastavenia dosiahli mierne nižšie hodnotenia, stále prevažne 4–5/5, avšak s ojedinelým 3/5, čo naznačuje priestor na zlepšenie v jasnosti a spoľahlivosti.
- **Kvalitatívne:** Účastníci oceňovali jednoduchosť prototypu a bezprostrednosť interakcií. Ako najviac pozitívna časť sa najčastejšie uvádzalo rozbaľovacie menu; časť účastníkov vyzdvihla aj nastavenia. Ako najmenej obľúbené sa opakovane objavili príklady detekcie a mini-modálne okno spätnej väzby. Najvýraznejšia negatívna spätná väzba sa týkala ovládania v nastave-

veniach, najmä poznámok o nefunkčných alebo ťažko použiteľných tlačidlách.

- **Faktory ovplyvnenia:** Malá a potenciálne homogénna vzorka, napríklad prevažne technicky zdatní účastníci, môže viesť k optimistickjším výsledkom. Tento faktor bol zohľadnený pri návrhu zmien pre ďalšiu iteráciu.

Podrobná tabuľka výsledkov dotazníka a súhrn pozorovaní sú dostupné v dokumentácii.¹

5.2.2 Druhý prototyp a jeho verifikácia

Druhý prototyp implementoval overený interakčný vzor v bežiacom HTML/React prototype (Vite + React). Táto iterácia zvýšila fidelitu a zabezpečila, že všetky prepínače a stavy sa správajú konzistentne: globálne zapnutie ochrany, správanie Auto-blur, zapnutie alebo vypnutie kategórií a voliteľné zobrazenie istoty.

Verifikácia v druhej iterácii opätovne použila rovnaký typ scenárových úloh, aby bolo možné porovnať zmeny medzi iteráciami. Dotazník bol administrovaný znovu a súhrnné skóre za druhý prototyp je **86.4/100**.

Interpretácia výsledkov. Oproti prvej iterácii sa zlepšila konzistentnosť stavov a objaviteľnosť ovládacích prvkov, najmä po presune kontextových akcií do hlavičky karty a po zjednotení vizuálnych stavov v celom rozhraní. Výsledky naznačujú, že druhá iterácia lepšie napĺňa požiadavku na rýchlu a reverzibilnú interakciu bez výrazného narušenia čítania.

Obmedzenia a faktory ovplyvňujúce výsledky. Malá alebo homogénna vzorka, napríklad prevažne technicky zdatní účastníci, môže skresliť výsledky hodnotenia. Zároveň druhý prototyp používa simulované výstupy detekcie, čo môže ovplyvniť vnímanú užitočnosť v porovnaní s integráciou reálneho modelu.

5.3 Vyhodnotenie vzdialeného režimu inferencie

Vzdialený režim inferencie bol hodnotený ako alternatíva k lokálnemu spracovaniu textu, ktorá umožňuje použiť výkonnejší model a flexibilnejšie experimentovať s nastavením prahov. Táto časť sa zameriava najmä na vplyv nerovnováhy tried, porovnanie jednotlivých iterácií modelu a rozdiel medzi validačne optimalizovanými a produktovými prahmi. Výsledky slúžia na posúdenie toho, akú úlohu môže vzdialený režim zohrávať v hybridnej architektúre rozšírenia.

¹Odkaz na výsledky testovania: [User Testing Results \(Spreadsheet\)](#).

5.3.1 Nerovnováha tried a príprava dát

Pri viacštítkovej detekcii toxicity podľa taxonómie Jigsaw sa potvrdila výrazná nerovnováha tried. V baseline rozdelení dát Jigsaw2018 dosahuje napríklad štítok *threat* iba približne 0,30% pozitívnych príkladov v tréningu a 0,26% vo validácii, pričom *severe_toxic* sa pohybuje približne na úrovni 1%. Na zlepšenie učenia menšinových tried boli otestované oversampling konfigurácie pre vybrané štítky. Kľúčovým rozhodnutím bolo aplikovať oversampling iba na tréningovú množinu a validačnú množinu ponechať nezmenenú, aby metriky ostali porovnateľné.

Split	Riadky	toxic	severe	obscene	insult	threat	identity
Baseline (train)	143 613	9,59%	1,01%	5,31%	4,94%	0,30%	0,89%
Baseline (val)	15 958	9,56%	0,90%	5,18%	4,88%	0,26%	0,83%
Oversampling (train-only, train)	152 294	14,56%	4,48%	9,63%	9,09%	2,55%	2,66%
Oversampling (train-only, val)	15 958	nezmenené (identické s Baseline (val))					

Tabuľka 5.1: Rozdelenie pozitívnych príkladov v Jigsaw2018 (baseline) a po oversamplingu aplikovanom iba na tréningovú množinu.

5.3.2 Iterácie modelu a porovnanie verzií

Na pripravených dátach boli vytrénované a vyhodnotené tri iterácie modelu XLM-R, konkrétne verzie v1, v2 a v2_1, pričom všetky boli hodnotené rovnakým protokolom na validačnej množine Jigsaw2018. Výsledky ukazujú, že verzia v2 dosiahla najlepší kompromis medzi makro-F1, mikro-F1 a Hammingovou stratou a zároveň výrazne zlepšila menšinové triedy. Príkladom je metrika *F1_threat*, ktorá vzrástla z 0,362 na 0,743.

Model	macro-F1	micro-F1	Hamming loss	F1_threat	F1_severe
xlmr-toxic-v1	0,635	0,777	0,0169	0,362	0,470
xlmr-toxic-v2	0,780	0,839	0,0122	0,743	0,628
xlmr-toxic-v2_1	0,647	0,768	0,0176	0,448	0,489

Tabuľka 5.2: Porovnanie iterácií XLM-R modelu na validačnej množine Jigsaw2018 (multi-label).

Verzia v2_1 bola použitá ako doplnková kontrolná konfigurácia pri experimentoch zameraných na prahovanie a stabilitu správania v produktových nastaveniach rozšírenia.

5.3.3 Prahy rozhodovania: tuned vs. product

Keďže ide o multi-label úlohu, nepostačuje jeden globálny prah. Preto boli zavedené per-label prahey v dvoch režimoch: *tuned* prahey optimalizované na validačnej množine a *product* prahey určené pre reálne správanie doplnku, ktoré zohľadňujú aj UX kompromisy. Pri bezpečnostne kritických prejavoch, ako je *threat*, bolo cieľom znížiť počet falošne negatívnych prípadov aj za cenu citlivejšieho rozhodovania.

Štítok	tuned	product
toxic	0,675	0,80
severe_toxic	0,95	0,70
obscene	0,89	0,92
insult	0,85	0,85
threat	0,945	0,55
identity_hate	0,95	0,75

Tabuľka 5.3: Porovnanie per-label prahov pre režim *tuned* (optimalizácia na val) a *product* (UX kompromis) pre v2_1.

Interpretácia. Výsledky ukazujú, že rozhodovanie modelu je pri menšinových a bezpečnostne citlivých triedach výrazne závislé od nastavenia prahov.

Z tohto dôvodu je vhodné oddelovať prahy optimalizované na validačných dátach od prahov určených pre produktové správanie systému, kde sa okrem klasickej presnosti zohľadňuje aj používateľský dopad falošných pozitív a falošných negatív.

5.4 Vyhodnotenie predtrénovaného TF.js toxicity modelu

Pred návrhom vlastného optimalizovaného lokálneho modelu bol orientačne vyhodnotený aj predtrénovaný model `@tensorflow-models/toxicity`. Tento experiment slúžil ako lokálny baseline a jeho cieľom bolo overiť, či je hotový TF.js model použiteľný pre režim orientovaný na súkromie používateľa. Model nebol použitý ako finálny lokálny režim riešenia, ale poskytol užitočné porovnanie z hľadiska kvality detekcie a výpočtovej náročnosti.

5.4.1 Nastavenie experimentu

Vyhodnotenie bolo vykonané pomocou samostatného Node.js testera, ktorý načítal validačný súbor `val.csv`, spustil predikciu modelu a uložil výsledky do súborov `summary_metrics.csv`, `per_label_metrics.csv`, `predictions.csv`, `config.json` a `run_log.txt`. Použitá bola verzia `@tensorflow-models/toxicity` 1.2.2 spolu s kompatibilnou verziou TensorFlow.js 1.7.4. Model bol testovaný s backendom *CPU*, globálnym rozhodovacím prahom $\tau = 0,85$ a veľkosťou dávky 16.

Keďže praktické lokálne nasadenie rozšírenia nepočítalo so spoľahlivým využitím GPU/WebGL akcelerácie, dôležité bolo najmä správanie modelu v lokálnom režime bez GPU. Toto nastavenie zároveň ukázalo, do akej miery je predtrénovaný TF.js model vhodný pre priebežné spracovanie väčšieho počtu komentárov.

Model	Backend	TF.js ver.	Riadky	Prahovanie	Batch size
@tensorflow-models/toxicity 1.2.2	CPU	1.7.4	15 958	$\tau = 0,85$	16
Metriky: macro-F1 = 0,279; micro-F1 = 0,560; Hamming loss = 0,0233; exact match accuracy = 0,908.					
Čas: načítanie modelu = 4,76 s; vyhodnotenie = 11 652,35 s; celkový čas = 11 657,22 s; priemerne približne 730 ms na jeden komentár.					

Tabuľka 5.4: Súhrn vyhodnotenia predtrénovaného TF.js toxicity modelu na validačnej množine Jigsaw2018.

5.4.2 Výsledky a interpretácia

Výsledky ukazujú, že model dosahoval najlepšie výsledky pri častejších kategóriách *toxic* a *insult*. Pri štítku *toxic* dosiahol F1 hodnotu 0,685 a pri štítku *insult* hodnotu 0,603. Pri kategórii *obscene* bola presnosť vysoká, ale návratnosť nízka, čo znamená, že model označoval túto kategóriu opatrne a veľkú časť pozitívnych príkladov neidentifikoval.

Pri zriedkavejších triedach *severe_toxic*, *threat* a *identity_hate* model pri zvolenom globálnom prahu neoznačil žiadny pozitívny príklad. Z toho dôvodu bola hodnota F1 pre tieto triedy nulová. Vysoké hodnoty accuracy pri týchto triedach preto nemožno interpretovať ako dôkaz kvalitnej detekcie, keďže sú spôsobené najmä veľkým počtom negatívnych príkladov vo validačných dátach.

Štítok	Pozit.	Predik. pozit.	Precision	Recall	F1
toxic	1 525	888	0,930	0,542	0,685
severe_toxic	143	0	0,000	0,000	0,000
obscene	826	211	0,948	0,242	0,386
insult	778	518	0,755	0,503	0,603
threat	42	0	0,000	0,000	0,000
identity_hate	132	0	0,000	0,000	0,000

Tabuľka 5.5: Per-label výsledky predtrénovaného TF.js toxicity modelu pri globálnom prahu $\tau = 0,85$.

Z praktického hľadiska bol najvýraznejším obmedzením čas spracovania. Vyhodnotenie 15 958 komentárov trvalo viac než tri hodiny, čo zodpovedá približne 730 ms na jeden komentár. Takéto správanie nie je vhodné pre rozšírenie prehli-

dača, ktoré musí priebežne spracovávať viacero textových uzlov na stránke bez cititeľného spomalenia používateľského rozhrania.

Na základe týchto výsledkov nebol predtrénovaný TF.js toxicity model použitý ako finálny lokálny režim. Experiment však ukázal dôležitý baseline: hotový TF.js model poskytuje určitú kvalitu pri častejších kategóriách, ale pri jednom globálnom prahu zlyháva pri menšinových triedach a jeho výpočtová náročnosť je pre praktické lokálne nasadenie príliš vysoká. Toto zistenie motivovalo návrh vlastného optimalizovaného lokálneho modelu, ktorý je opísaný v ďalšej časti.

5.5 Vyhodnotenie optimalizovaného lokálneho režimu inferencie

Po vyhodnotení pôvodného lokálneho baseline modelu bol lokálny režim rozšírený o vlastný optimalizovaný model vo formáte TensorFlow.js GraphModel. Táto časť hodnotí, či takýto model dokáže zlepšiť klasifikačnú kvalitu, zachovať výhody lokálneho spracovania a zároveň dosiahnuť dostatočný výkon pre praktické použitie v rozšírení. Pozornosť je venovaná ladeniu prahov, výsledkom na validačných dátach a porovnaniu backendov CPU a WASM.

5.5.1 Nastavenie experimentu

Táto časť nadväzuje na predchádzajúce experimentálne vyhodnotenie lokálneho TF.js modelu založeného na predtrénovanom balíku toxicity modelu. Pôvodné riešenie slúžilo ako orientačný lokálny baseline, avšak pri jednom globálnom prahu dosahovalo slabšie výsledky najmä pri menšinových triedach. Z tohto dôvodu bol lokálny režim ďalej rozšírený o vlastný model exportovaný do formátu TensorFlow.js GraphModel.

Nový lokálny model pracuje na znakovnej reprezentácii vstupu. Každý text je normalizovaný, skrátený na maximálnu dĺžku 256 znakov a následne zakódovaný podľa slovníka znakov. Výstupom modelu je šesťica skóre zodpovedajúca taxonómii Jigsaw: *toxic*, *severe_toxic*, *obscene*, *insult*, *threat* a *identity_hate*.

Cieľom experimentu bolo overiť dve vlastnosti optimalizovaného lokálneho režimu. Prvou bola klasifikačná použiteľnosť modelu na validačnej množine Jigsaw2018. Druhou bolo porovnanie výkonu backendov CPU a WASM, keďže lokálna inferencia v rozšírení prehliadača musí byť dostatočne rýchla, aby nespomaľovala používateľské rozhranie. Experiment bol vykonaný na validačnej množine *val.csv* s počtom 15 958 komentárov. Veľkosť dávky bola nastavená na 64

a použitá verzia TensorFlow.js bola 4.22.0.

Konfigurácia	Model	TF.js ver.	Riadky	Batch size	Max. dĺžka vstupu
Optimalizovaný lokálny režim	Char-CNN GraphModel	4.22.0	15 958	64	256 znakov

Tabuľka 5.6: Nastavenie experimentu pre optimalizovaný lokálny režim inferencie pomocou TensorFlow.js GraphModel.

5.5.2 Ladenie rozhodovacích prahov

Na rozdiel od pôvodného lokálneho baseline riešenia, ktoré bolo vyhodnotené s jedným globálnym prahom, nový lokálny model používa samostatné rozhodovacie prahy pre jednotlivé štítky. Takéto nastavenie lepšie zodpovedá viacštítkovej povahe úlohy, pretože jednotlivé kategórie toxicity majú rozdielne rozdelenie dát a rozdielnu náročnosť detekcie.

Prahy boli optimalizované na validačnej množine tak, aby sa maximalizovala hodnota F1 pre jednotlivé triedy. Výsledky ukazujú, že model generuje pomerne vysoké skóre pri pozitívnych prípadoch, preto sa najlepšie prahy pohybovali v rozsahu 0,90 až 0,94.

Režim	toxic	severe	obscene	insult	threat	identity
Tuned	0,90	0,94	0,94	0,92	0,94	0,94
Product	0,90	0,96	0,94	0,92	0,97	0,96

Tabuľka 5.7: Porovnanie per-label prahov pre optimalizovaný lokálny model. Režim *tuned* maximalizuje F1 na validačnej množine, zatiaľ čo režim *product* sprísňuje rozhodovanie pri zriedkavých triedach.

Režim *product* bol navrhnutý ako konzervatívnejšie nastavenie pre reálne použitie v rozšírení. Pri triedach *severe_toxic*, *threat* a *identity_hate* boli prahy zvýšené, pretože tieto triedy majú výrazne menší počet pozitívnych príkladov a pri optimalizácii iba podľa F1 vykazovali nižšiu presnosť. Takéto nastavenie znižuje riziko falošne pozitívneho rozmazania bežného textu.

5.5.3 Klasifikačné výsledky na validačnej množine

Klasifikačné výsledky ukazujú, že model dosahuje najlepšie výsledky pri častejších triedach, najmä *toxic*, *obscene* a *insult*. Najvyššiu hodnotu F1 dosiahol štítok *obscene* s hodnotou 0,801. Trieda *toxic* dosiahla F1 0,732 a *insult* 0,702. Slabšie výsledky boli pozorované pri zriedkavých triedach, hlavne *threat*, kde bolo vo validačnej množine iba 42 pozitívnych príkladov.

Štítok	Pozit.	Prah	Precision	Recall	F1	Accuracy
toxic	1 525	0,90	0,787	0,684	0,732	0,952
severe_toxic	143	0,94	0,286	0,783	0,419	0,981
obscene	826	0,94	0,808	0,794	0,801	0,980
insult	778	0,92	0,664	0,743	0,702	0,969
threat	42	0,94	0,189	0,714	0,299	0,991
identity_hate	132	0,94	0,329	0,636	0,434	0,986

Tabuľka 5.8: Per-label výsledky optimalizovaného lokálneho modelu na validačnej množine Jigsaw2018.

Okrem samostatných štítkov bol vyhodnotený aj celkový binárny verdikt *toxic/non-toxic*, ktorý zodpovedá praktickému správaniu rozšírenia. Text je v používateľskom rozhraní označený alebo rozmazaný vtedy, ak aspoň jeden povolený štítok prekročí nastavený prah. Pri tomto zjednodušenom rozhodovaní dosiahol model F1 0,734 pri najlepšom globálnom prahu 0,86.

Verdikt	Pozit.	Prah	Precision	Recall	F1	Accuracy
toxic / non-toxic	1 623	0,86	0,744	0,725	0,734	0,947

Tabuľka 5.9: Vyhodnotenie celkového binárneho verdiktu lokálneho modelu na validačnej množine Jigsaw2018.

Interpretácia. Výsledky potvrdzujú, že optimalizovaný lokálny model je výrazne použiteľnejší ako pôvodný lokálny baseline opísaný v predchádzajúcej časti. Zatiaľ čo pôvodný model bol vhodný najmä ako orientačné riešenie typu *privacy-friendly baseline*, nový GraphModel poskytuje skóre pre všetkých šesť štítkov v rovnakej taxonómii ako vzdialený režim a umožňuje samostatné prahovanie pre každú kategóriu. Napriek tomu zostáva detekcia menšinových tried náročná, najmä pre *threat* a *identity_hate*, kde nízky počet pozitívnych príkladov zvyšuje citlivosť metriky precision na falošne pozitívne predikcie.

5.5.4 Porovnanie výkonu backendov CPU a WASM

Dôležitou súčasťou lokálneho režimu je výkon inferencie. Model bol preto spustený s dvomi backendmi TensorFlow.js: *CPU* a *WASM*. Pri oboch backendch boli klasifikačné metriky rovnaké, rozdiel sa prejavil najmä v rýchlosti výpočtu. Backend *CPU* spracoval 15 958 komentárov za 85,60 s, zatiaľ čo backend *WASM* potreboval 6,35 s. Priemerný čas na jeden komentár klesol z 5,36 ms na 0,40 ms.

Backend	Riadky	Batch	Predikcia [s]	Priemer [ms]	Priepust. [textov/s]	Warm-up [ms]
CPU	15 958	64	85,60	5,36	186,42	34,27
WASM	15 958	64	6,35	0,40	2 514,57	10,66

Tabuľka 5.10: Porovnanie výkonu lokálnej inferencie pri backendch CPU a WASM.

Backend *WASM* bol približne 13,49-krát rýchlejší než backend *CPU* pri samotnej predikcii. Tento rozdiel je prakticky významný, pretože rozšírenie neklasifikuje iba jeden text, ale priebežne skenuje viaceré textové uzly na stránke. Rýchlejšia inferencia znižuje riziko oneskorenia pri prehliadaní obsahu a umožňuje spracovanie väčšieho počtu komentárov bez výrazného dopadu na používateľské rozhranie.

5.6 Zhrnutie vyhodnotenia

Vyhodnotenie ukázalo, že navrhnuté používateľské rozhranie je zrozumiteľné a použiteľné. Druhá iterácia prototypu dosiahla lepšie výsledky než prvá a odstránila viaceré problémy zistené pri prvotnom testovaní.

Z technického hľadiska sa potvrdilo, že lokálny a vzdialený režim inferencie majú rozdielne výhody. Optimalizovaný lokálny režim zachováva princíp *privacy-by-default*, keďže text sa spracúva priamo v prehliadači, a pri použití backendu *WASM* dosahuje dostatočnú rýchlosť na praktické použitie. V porovnaní s pôvodným baseline riešením poskytuje vlastný *GraphModel* lepšiu kontrolu nad prahmi, jednotnejšiu štruktúru výstupu a lepšie možnosti optimalizácie.

Lokálny TF.js model je preto vhodný ako základ pre súkromie orientované nasadenie, avšak pri jednoduchom globálnom prahu dosahuje slabšie výsledky pri menej zastúpených kategóriách. Vzdialený režim naopak umožňuje lepšiu optimalizáciu presnosti, najmä pri menšinových triedach a *per-label* prahovaní.

Navrhnutá hybridná architektúra tak predstavuje vhodný kompromis medzi ochranou súkromia, použiteľnosťou, výkonom a presnosťou detekcie toxického textu.

6 Záver

Cieľom tejto záverečnej práce bolo preskúmať možnosti detekcie toxického textu v prostredí webového prehliadača a navrhnúť, implementovať a vyhodnotiť riešenie vo forme rozšírenia *Toxic Text Detector*. Práca sa zamerala na viacštítkovú klasifikáciu toxicity, používateľské rozhranie, ochranu súkromia, architektúru rozšírenia podľa Chrome Manifest V3 a porovnanie lokálneho a vzdialeného režimu inferencie. Výsledkom je funkčný prototyp hybridného systému, ktorý kombinuje spracovanie textu priamo v prehliadači s možnosťou vzdialenej inferencie.

V analytickej časti boli spracované teoretické východiská detekcie toxicity, relevantné datasety, modely, existujúce prístupy a technické aj etické problémy tejto oblasti. Analýza ukázala, že pri návrhu podobného systému nie je rozhodujúca iba presnosť modelu, ale aj spôsob nasadenia, latencia, transparentnosť výsledkov, ochrana súkromia a používateľská dôvera. Na základe týchto poznatkov bolo navrhnuté a implementované rozšírenie pozostávajúce z používateľského rozhrania v Reacte, service workera, content scriptu a vrstvy nastavení uchováanej v úložisku prehliadača. Rozšírenie umožňuje analyzovať vybraný text alebo textové časti stránky, označiť potenciálne toxický obsah, vizuálne ho rozmazať a poskytnúť používateľovi základné vysvetlenie výsledku pomocou kategórie a skóre.

Dôležitou súčasťou práce bol aj návrh používateľského rozhrania. Rozhranie bolo vytvorené iteratívne, najskôr ako prototyp vo Figma a následne ako bežiaci HTML/React prototyp. Používateľské testovanie ukázalo, že zvolený koncept je zrozumiteľný a používatelia pozitívne vnímali najmä jednoduchosť ovládania, možnosť spätne zobrazíť skrytý text a jasné signalizovanie stavu ochrany. Mechanizmus spätnej väzby používateľa pri nesprávnej klasifikácii bol zvažovaný v skorom návrhu, ale keďže išlo o voliteľné rozšírenie a jeho dopracovanie by si vyžadovalo ďalší návrh, ukladanie údajov a samostatné testovanie, nebol zahrnutý do finálnej implementácie.

V oblasti inferencie práca ukázala význam hybridného prístupu. Pôvodne bol

ako lokálny variant overený predtrénovaný TF.js model toxicity, no jeho testovanie ukázalo obmedzenia z hľadiska rýchlosti, aktuálnosti a kvality výsledkov pri menej zastúpených kategóriách. Preto bol lokálny režim rozšírený o optimalizovaný TensorFlow.js GraphModel so samostatnými prahmi pre jednotlivé štítky a efektívnejším behom cez WASM backend. Lokálny režim sa ukázal ako rýchly a súkromie zachovávajúci základ, avšak s nižšou presnosťou pri náročnejších kategóriách. Vzdialený režim poskytol priestor pre presnejšiu detekciu a jednoduchšiu výmenu modelu, pričom pri lokálnom spustení API na výkonnom notebooku dosahoval odozvu porovnateľnú s lokálnym režimom. Vyhodnotenie zároveň potvrdilo, že pri detekcii toxicity je vhodné používať samostatné prahy pre jednotlivé kategórie namiesto jedného univerzálneho prahu.

Z pohľadu splnenia cieľov možno konštatovať, že hlavné ciele práce boli splnené. Bola spracovaná analytická časť, navrhnutá architektúra rozšírenia, vytvorené a otestované používateľské rozhranie, implementovaná funkčná verzia rozšírenia pre Chrome Manifest V3 a pripravený lokálny aj vzdialený režim inferencie. Práca zároveň ukázala obmedzenia riešenia, najmä pri detekcii implicitnej, kontextovej a sarkastickej toxicity, pri menej zastúpených kategóriách, pri viacjazyčných textoch a pri dôkladnom hodnotení spravodlivosti modelov. Ďalší vývoj by sa preto mohol zamerať na presnejšiu kalibráciu prahov, rozšírenie vzdialeného režimu o ďalšie modely, lepšie spracovanie rôznych jazykov, používateľskú spätnú väzbu a rozsiahlejšie testovanie na reálnych webových stránkach. Navrhnuté riešenie tak potvrdilo realizovateľnosť prehliadačovo integrovaného nástroja, ktorý vyvažuje presnosť detekcie, používateľskú kontrolu a ochranu súkromia.

Zoznam použitej literatúry

1. DAVIDSON, Thomas; WARMSLEY, Dana; MACY, Michael; WEBER, Ingmar. Automated Hate Speech Detection and the Problem of Offensive Language. In: *Proceedings of the International AAAI Conference on Web and Social Media*. AAAI Press, 2017, zv. 11, s. 512–515. Č. 1. Dostupné tiež z: <https://ojs.aaai.org/index.php/ICWSM/article/view/14955>.
2. WASEEM, Zeerak; HOVY, Dirk. Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter. In: *Proceedings of the NAACL Student Research Workshop*. San Diego, California: Association for Computational Linguistics, 2016, s. 88–93. Dostupné z doi: [10.18653/v1/N16-2013](https://doi.org/10.18653/v1/N16-2013).
3. FOUNTA, Antigoni-Maria; DJOUVAS, Constantinos; CHATZAKOU, Despoina; LEONTIADIS, Ilias; BLACKBURN, Jeremy; STRINGHINI, Gianluca; VAKALI, Athena; SIRIVIANOS, Michael; KOURTELLIS, Nicolas. Large Scale Crowdsourcing and Characterization of Twitter Abusive Behavior. In: *Proceedings of the International AAAI Conference on Web and Social Media*. AAAI Press, 2018, zv. 12, s. 491–500. Č. 1. Dostupné tiež z: <https://ojs.aaai.org/index.php/ICWSM/article/view/14991>.
4. ZAMPIERI, Marcos; MALMASI, Shervin; NAKOV, Preslav; ROSENTHAL, Sara; FARRA, Noura; KUMAR, Ritesh. SemEval-2019 Task 6: Identifying and Categorizing Offensive Language in Social Media (OffensEval). In: *Proceedings of the 13th International Workshop on Semantic Evaluation*. Minneapolis, Minnesota, USA: Association for Computational Linguistics, 2019, s. 75–86. Dostupné z doi: [10.18653/v1/S19-2010](https://doi.org/10.18653/v1/S19-2010).
5. TSOUMAKAS, Grigorios; KATAKIS, Ioannis. Multi-label Classification: An Overview. *International Journal of Data Warehousing and Mining*. 2007, roč. 3, č. 3, s. 1–13. Dostupné z doi: [10.4018/jdwm.2007070101](https://doi.org/10.4018/jdwm.2007070101).
6. JIGSAW. *Jigsaw Toxic Comment Classification Challenge* [Kaggle competition]. 2018. Dostupné tiež z: <https://www.kaggle.com/competitions/jig>

- saw-toxic-comment-classification-challenge. Citované: 2026-05-16.
7. JIGSAW. *Jigsaw Unintended Bias in Toxicity Classification* [Kaggle competition]. 2019. Dostupné tiež z: <https://www.kaggle.com/competitions/jigsaw-unintended-bias-in-toxicity-classification>. Citované: 2026-05-16.
8. JIGSAW. *Jigsaw Multilingual Toxic Comment Classification* [Kaggle competition]. 2020. Dostupné tiež z: <https://www.kaggle.com/competitions/jigsaw-multilingual-toxic-comment-classification>. Citované: 2026-05-16.
9. KUMAR, Sharath; DMELLO, Daxia Vlor; JANANI, Devi; K. D., Avani; P., Kirthana Kamath. Eliminating Toxicity in Text: An NLP Framework for Clean Content Extraction. In: *2024 International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS)*. IEEE, 2024, s. 146–151. Dostupné z DOI: [10.1109/ICICNIS64247.2024.10823171](https://doi.org/10.1109/ICICNIS64247.2024.10823171).
10. SIKIANDANI, Ni Made Deni; SUARJAYA, I. Made Agus Dwi; PUTRA, Yohanes Perdana. Browser-Based Detection of Harmful Content with Deep Learning Model. *Journal of Applied Informatics and Computing*. 2025, roč. 9, č. 4, s. 1800–1811. Dostupné z DOI: [10.30871/jaic.v9i4.9804](https://doi.org/10.30871/jaic.v9i4.9804).
11. DEVLIN, Jacob; CHANG, Ming-Wei; LEE, Kenton; TOUTANOVA, Kristina. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, s. 4171–4186. Dostupné z DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423).
12. LIU, Yinhan; OTT, Myle; GOYAL, Naman; DU, Jingfei; JOSHI, Mandar; CHEN, Danqi; LEVY, Omer; LEWIS, Mike; ZETTLEMOYER, Luke; STOYANOV, Veselin. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*. 2019. Dostupné z DOI: [10.48550/arXiv.1907.11692](https://doi.org/10.48550/arXiv.1907.11692).
13. DEEP, Shashank; SINGH, Sahil; YADAV, Shubham; SHEDGE, Yash; KHADDE, Anindita. Creating an Integrated Browser Plug-in for Detecting and Blocking Obscene Image/Text Content. In: *2024 2nd International Conference on Networking and Communications (ICNWC)*. IEEE, 2024, s. 1–6. Dostupné z DOI: [10.1109/ICNWC60771.2024.10537464](https://doi.org/10.1109/ICNWC60771.2024.10537464).

14. JIGSAW. *Perspective API: Attributes, Models and Language Support* [Perspective API Documentation]. 2026. Dostupné tiež z: <https://developers.perspectiveapi.com/>. Citované: 2026-05-16.
15. TENSORFLOW.JS TEAM. *TensorFlow.js Models: Toxicity Classifier* [GitHub repository]. 2026. Dostupné tiež z: <https://github.com/tensorflow/tfjs-models/tree/master/toxicity>. Citované: 2026-05-16.
16. GOOGLE CHROME DEVELOPERS. *Manifest V3 Overview* [Chrome Extensions Documentation]. 2026. Dostupné tiež z: <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3>. Citované: 2026-05-16.
17. EUROPEAN DATA PROTECTION BOARD. *Guidelines 4/2019 on Article 25 Data Protection by Design and by Default* [European Data Protection Board]. 2020. Dostupné tiež z: https://www.edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-42019-article-25-data-protection-design-and_en. Citované: 2026-05-16.
18. MATHEW, Binny; SAHA, Punyajoy; YIMAM, Seid Muhie; BIEMANN, Chris; GOYAL, Pawan; MUKHERJEE, Animesh. HateXplain: A Benchmark Dataset for Explainable Hate Speech Detection. *Proceedings of the AAAI Conference on Artificial Intelligence*. 2021, roč. 35, č. 17, s. 14867–14875. Dostupné z DOI: [10.1609/aaai.v35i17.17745](https://doi.org/10.1609/aaai.v35i17.17745).
19. BORKAN, Daniel; DIXON, Lucas; SORENSEN, Jeffrey; THAIN, Nithum; VASSERMAN, Lucy. Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. In: *Companion Proceedings of The 2019 World Wide Web Conference*. ACM, 2019, s. 491–500. Dostupné z DOI: [10.1145/3308560.3317593](https://doi.org/10.1145/3308560.3317593).
20. RADFORD, Alec; KIM, Jong Wook; XU, Tao; BROCKMAN, Greg; MCLEAVEY, Christine; SUTSKEVER, Ilya. Robust Speech Recognition via Large-Scale Weak Supervision. *arXiv preprint arXiv:2212.04356*. 2022. Dostupné z DOI: [10.48550/arXiv.2212.04356](https://doi.org/10.48550/arXiv.2212.04356).
21. DIXON, Lucas; LI, John; SORENSEN, Jeffrey; THAIN, Nithum; VASSERMAN, Lucy. Measuring and Mitigating Unintended Bias in Text Classification. In: *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*. ACM, 2018, s. 67–73. Dostupné z DOI: [10.1145/3278721.3278729](https://doi.org/10.1145/3278721.3278729).

22. SAP, Maarten; CARD, Dallas; GABRIEL, Saadia; CHOI, Yejin; SMITH, Noah A. The Risk of Racial Bias in Hate Speech Detection. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, 2019, s. 1668–1678. Dostupné z DOI: [10.18653/v1/P19-1163](https://doi.org/10.18653/v1/P19-1163).
23. HARTVIGSEN, Thomas; GABRIEL, Saadia; PALANGI, Hamid; SAP, Maarten; RAY, Dipankar; KAMAR, Ece. ToxiGen: A Large-Scale Machine-Generated Dataset for Adversarial and Implicit Hate Speech Detection. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, 2022, s. 3309–3326. Dostupné z DOI: [10.18653/v1/2022.acl-long.234](https://doi.org/10.18653/v1/2022.acl-long.234).
24. RÖTTGER, Paul; VIDGEN, Bertie; NGUYEN, Dong; WASEEM, Zeerak; MARGETTS, Helen; PIERREHUMBERT, Janet. HateCheck: Functional Tests for Hate Speech Detection Models. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*. Online: Association for Computational Linguistics, 2021, s. 41–58. Dostupné z DOI: [10.18653/v1/2021.acl-long.4](https://doi.org/10.18653/v1/2021.acl-long.4).
25. BOJANOWSKI, Piotr; GRAVE, Edouard; JOULIN, Armand; MIKOLOV, Tomas. Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*. 2017, roč. 5, s. 135–146. Dostupné z DOI: [10.1162/tac1_a_00051](https://doi.org/10.1162/tac1_a_00051).
26. MIKOLOV, Tomas; CHEN, Kai; CORRADO, Greg; DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. 2013. Dostupné z DOI: [10.48550/arXiv.1301.3781](https://doi.org/10.48550/arXiv.1301.3781).
27. PENNINGTON, Jeffrey; SOCHER, Richard; MANNING, Christopher D. GloVe: Global Vectors for Word Representation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. Doha, Qatar: Association for Computational Linguistics, 2014, s. 1532–1543. Dostupné z DOI: [10.3115/v1/D14-1162](https://doi.org/10.3115/v1/D14-1162).
28. SANH, Victor; DEBUT, Lysandre; CHAUMOND, Julien; WOLF, Thomas. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv preprint arXiv:1910.01108*. 2019. Dostupné z DOI: [10.48550/arXiv.1910.01108](https://doi.org/10.48550/arXiv.1910.01108).

29. CONNEAU, Alexis; KHANDELWAL, Kartikay; GOYAL, Naman; CHAUDHARY, Vishrav; WENZKE, Guillaume; GUZMÁN, Francisco; GRAVE, Edouard; OTT, Myle; ZETTLEMOYER, Luke; STOYANOV, Veselin. Unsupervised Cross-lingual Representation Learning at Scale. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, 2020, s. 8440–8451. Dostupné z DOI: [10.18653/v1/2020.acl-main.747](https://doi.org/10.18653/v1/2020.acl-main.747).
30. LIN, Tsung-Yi; GOYAL, Priya; GIRSHICK, Ross; HE, Kaiming; DOLLÁR, Piotr. Focal Loss for Dense Object Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2020, roč. 42, č. 2, s. 318–327. Dostupné z DOI: [10.1109/TPAMI.2018.2858826](https://doi.org/10.1109/TPAMI.2018.2858826).
31. SCIKIT-LEARN DEVELOPERS. *Precision-Recall* — *scikit-learn Documentation* [scikit-learn Documentation]. 2026. Dostupné tiež z: https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html. Citované: 2026-05-16.
32. GUO, Chuan; PLEISS, Geoff; SUN, Yu; WEINBERGER, Kilian Q. On Calibration of Modern Neural Networks. In: *Proceedings of the 34th International Conference on Machine Learning*. PMLR, 2017, s. 1321–1330. Dostupné tiež z: <https://proceedings.mlr.press/v70/guo17a.html>.
33. SCIKIT-LEARN DEVELOPERS. *f1_score* — *scikit-learn Documentation* [scikit-learn Documentation]. 2026. Dostupné tiež z: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html. Citované: 2026-05-16.
34. SCIKIT-LEARN DEVELOPERS. *hamming_loss* — *scikit-learn Documentation* [scikit-learn Documentation]. 2026. Dostupné tiež z: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.hamming_loss.html. Citované: 2026-05-16.
35. GOOGLE CHROME DEVELOPERS. *Manifest: Content Security Policy* [Chrome Extensions Documentation]. 2026. Dostupné tiež z: <https://developer.chrome.com/docs/extensions/reference/manifest/content-security-policy>. Citované: 2026-05-16.
36. GOOGLE CHROME DEVELOPERS. *Migrate to a Service Worker* [Chrome Extensions Documentation]. 2026. Dostupné tiež z: <https://developer.chrome.com/docs/extensions/develop/migrate/to-service-workers>. Citované: 2026-05-16.

37. GOOGLE CHROME DEVELOPERS. *Content Scripts* [Chrome Extensions Documentation]. 2026. Dostupné tiež z: <https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts>. Citované: 2026-05-16.
38. GOOGLE CHROME DEVELOPERS. *Message Passing* [Chrome Extensions Documentation]. 2026. Dostupné tiež z: <https://developer.chrome.com/docs/extensions/develop/concepts/messaging>. Citované: 2026-05-16.

Zoznam príloh

Príloha A CD médium – záverečná práca v elektronickej podobe:

- `doc` – záverečná práca a prílohy
- `tex` – zdrojové súbory záverečnej práce a vedeckého článku
- `src` – zdrojové kódy vytvoreného riešenia
- `dist` – zostavená verzia rozšírenia pripravená na načítanie do prehliadača
- `article` – vedecký článok vytvorený na základe výsledkov práce

Príloha B Systémová príručka

Príloha C Používateľská príručka

B Systémová príručka

Systémová príručka je súčasťou Git repozitára vo forme súboru README.md. Opisuje technické zloženie, inštaláciu, zostavenie, konfiguráciu a údržbu rozšírenia *Toxic Text Detector*. Je určená najmä pre vývojárov alebo používateľov, ktorí chcú aplikáciu spustiť zo zdrojových súborov, upraviť jej funkcionality alebo pokračovať vo vývoji riešenia.

Systémová príručka je dostupná aj online.²

Projekt je rozdelený na dve hlavné časti. Priečink *frontend* obsahuje klientsku časť systému vo forme rozšírenia prehliadača Chrome. Priečink *backend* obsahuje voliteľnú serverovú časť určenú na vzdialenú inferenciu toxicity cez API.

Príručka opisuje štruktúru projektu, použité technológie, lokálny a vzdialený model, zostavenie rozšírenia, načítanie do prehliadača, spustenie API servera, konfiguráciu vzdialeného API a pomocné skripty použité pri hodnotení modelov.

²Systémová príručka dostupná online: <https://github.com/kkuichi/vs800ru>.

C Používateľská príručka

Používateľská príručka je súčasťou Git repozitára. V repozitári je uložená ako súbor `USER_MANUAL.md`.³

Príručka je určená pre používateľa alebo hodnotiteľa, ktorý chce rozšírenie *Toxic Text Detector* nainštalovať a používať bez zásahu do zdrojového kódu. Opisuje inštaláciu cez súkromný odkaz v Chrome Web Store alebo manuálne načítanie pripravenej zložky `dist`.

Ďalej vysvetľuje používanie popup okna, nastavení, kategórií toxicity, automatického rozmazania, skóre istoty, zoznamu povolených domén a režimov lokálneho a vzdialeného spracovania. Súčasťou príručky sú aj odporúčané nastavenia, poznámky k ochrane súkromia, obmedzenia systému a základné riešenie problémov.

³Používateľská príručka dostupná online: https://github.com/kkuichi/vs800ru/blob/main/USER_MANUAL.md.